# Отчет о разработке игры ECHOES OF THE VOID

Подробное описание выполненной работы и текущего состояния проекта

Дата отчета 1 октября 2026 года

В проекте ECHOES OF THE VOID создан прототип трехмерной игры от первого лица на Godot Engine. Подготовлены сцены игрового пространства, главного меню и вступления, реализованы перемещение персонажа, обзор мышью, прыжок, управление фонарем и базовое взаимодействие с радиоприемником. Визуальную основу составляют импортированные трехмерные модели, контрастное освещение и экранный шейдер дизеринга. Основной технический результат работы — возможность исследовать помещение и переключаться из обычного управления персонажем в режим работы с прибором с плавным перемещением камеры.

Текущая сборка соответствует этапу прототипирования. Полный цикл поиска и расшифровки радиосигналов, шлюзование, выход в открытый космос, аварийные системы и сохранение прогресса еще не реализованы. В существующей последовательности запуска обнаружены ошибки настройки сцен и анимаций. Эти ограничения рассмотрены отдельно, чтобы объем выполненной работы можно было оценить по конкретным файлам, параметрам и результатам проверки.

# Содержание

1 Общая характеристика проекта и цель работы
2 Средства разработки и настройки проекта
3 Структура файлов и архитектура сцен
4 Трехмерные ресурсы и организация игрового пространства
5 Главное меню и переход к новой игре
6 Вступительная сцена
7 Контроллер персонажа от первого лица
8 Взаимодействие с игровыми объектами
9 Радиоприемник и режим работы с прибором
10 Освещение и управление фонарем
11 Шейдер дизеринга и визуальный стиль
12 Звуковые ресурсы
13 Проверка проекта и результаты испытаний
14 Соответствие постановке задачи
15 Обнаруженные ограничения и направления доработки
16 Итоги выполненной работы
Приложение А Полные листинги программных модулей
Приложение Б Полные листинги сцен и конфигурации
Приложение В Перечень скриншотов
Приложение Г Перечень исходных материалов

# 1 Общая характеристика проекта и цель работы

## 1 1 Назначение игры

ECHOES OF THE VOID задуман как психологический космический хоррор от первого лица. Игровая ситуация связана с исследованием аварийного челнока, взаимодействием с бортовым оборудованием и поиском аномальных радиосигналов. В постановке задачи предусмотрено расширение игрового процесса за счет диагностики неисправностей, ремонта и выхода в открытый космос. Текущий проект формирует основу внутреннего игрового пространства и управления персонажем.

Игра ориентирована на персональный компьютер. Взаимодействие строится на клавиатуре и мыши: игрок перемещается внутри помещения, изменяет направление взгляда, освещает объекты фонарем и активирует приборы. Камера от первого лица позволяет связать внимание игрока с конкретным предметом: объект взаимодействия определяется направленным лучом, исходящим из камеры.

## 1 2 Цель выполненной работы

Целью текущего этапа стало создание программной и визуальной основы проекта. Для этого подготовлены сцены, подключены трехмерные ресурсы, написан контроллер персонажа и разработан прототип взаимодействия с радиоприемником. Отдельно оформлены главное меню и вступительное пространство, а также реализован экранный эффект ограничения цветовой палитры.

Главный результат этого этапа состоит в соединении нескольких подсистем внутри одной сцены. Персонаж управляется через CharacterBody3D, камера размещена в иерархии головы, луч выбирает объект, а радиоприемник переводит камеру в заранее заданное положение. При переходе к прибору обычное движение блокируется; при завершении взаимодействия прежнее положение камеры и скорость восстанавливаются.

## 1 3 Границы реализованного прототипа

В проекте присутствуют шесть файлов GDScript, пять самостоятельных сцен TSCN, один экранный шейдер, четыре модели GLB и набор графических и звуковых ресурсов. Код программных модулей содержит 200 строк с учетом пустых строк и комментариев. Шейдер содержит еще 33 строки. Полные листинги всех этих файлов приведены в приложениях.

Прототип демонстрирует работу базовых механик, однако наличие объекта радио само по себе не означает наличие полноценной радиосистемы. В текущем коде есть вход в режим прибора и поворот ручки. Переменной частоты, логики резонанса, осциллограммы, сохранения пакета и терминала расшифровки пока нет. Аналогично, цветная аварийная подсветка создает визуальный образ корабля, но не реализует систему аварий и ремонта.

# 2 Средства разработки и настройки проекта

## 2 1 Движок и язык программирования

Основной инструмент разработки — Godot Engine. В project.godot указана ветка возможностей 4.7 и режим GL Compatibility. Проверка проекта выполнялась установленным Godot Engine 4.7.2 stable. Программная логика написана на GDScript с использованием типов, наследования, экспортируемых свойств и ссылок на узлы, получаемых через @onready.

Сцены сохранены в текстовом формате TSCN. Этот формат хранит состав узлов, трансформации, параметры света, ссылки на внешние ресурсы, встроенные материалы, анимации и соединения сигналов. Благодаря текстовому представлению можно проследить, какой скрипт назначен узлу, какая камера используется и какие обработчики подключены к кнопкам.

| Средство или формат | Применение в проекте |
|---|---|
| Godot Engine 4.7.2 | Загрузка проекта, исполнение сцен, физика, отображение трехмерного мира |
| GDScript | Управление игроком, переключение сцен, взаимодействие и работа радио |
| TSCN | Описание узлов, ресурсов, параметров и сигналов |
| GLB | Импорт геометрии и материалов трехмерных объектов |
| Blender и файлы BLEND | Хранение исходных файлов трехмерного контента в каталоге blender |
| Godot Shader Language | Экранный эффект дизеринга и квантования палитры |
| MP3 | Звуковые ресурсы вступительной сцены |
| Git | Хранение версий проекта и схем проектирования |

## 2 2 Параметры приложения

В конфигурации application имя приложения задано как test. Документация использует название ECHOES OF THE VOID, поэтому внутреннее имя приложения еще предстоит привести к названию проекта. Главной сценой указан UID uid://dqqbblk6vnnqr, соответствующий scenes/main.tscn. При обычном запуске проекта движок открывает уровень напрямую. Для демонстрации меню необходимо отдельно запускать main_menu.tscn либо изменить главную сцену в настройках проекта.

Базовый размер области отображения составляет 1680 × 1050. Режим растяжения canvas_items задан для двумерного содержимого. Соотношение сторон базового разрешения равно 16:10. В постановке задачи целевым разрешением названо 1920 × 1080, то есть 16:9; проверка интерфейса в этом разрешении является отдельной задачей дальнейшего этапа.

| Параметр project.godot | Текущее значение | Значение для работы |
|---|---|---|
| config/name | test | Внутреннее имя приложения |
| run/main_scene | UID сцены main.tscn | Прямой запуск основного уровня |
| viewport_width и viewport_height | 1680 и 1050 | Базовый размер области отображения |
| window/stretch/mode | canvas_items | Режим масштабирования двумерных элементов |
| 3d/physics_engine | Jolt Physics | Физическая подсистема трехмерной сцены |
| renderer/rendering_method | gl_compatibility | Используемый способ визуализации |
| import/blender/enabled | false | Прямой импорт BLEND отключен |
| textures/canvas_textures/default_texture_filter | 0 | Ближайшая фильтрация текстур CanvasItem |

### 2 3 Организация обработки кадров

Контроллер персонажа разделяет обработку ввода и физическое движение. События мыши и клавиатуры обрабатываются в _unhandled_input, а изменение скорости и вызов move_and_slide выполняются в _physics_process. Такое разделение делает поведение коллизий зависимым от физического шага, а направление взгляда — от поступающих событий мыши.

Вращение Земли выполняется в _process. Угловое изменение умножается на delta, поэтому вращение задано через время между кадрами. Для плавного перемещения камеры радио используется Tween: он изменяет глобальную трансформацию камеры на протяжении 0,5 секунды, а завершение перехода отслеживается через await tween.finished.

## 3 Структура файлов и архитектура сцен

### 3 1 Организация каталогов

Основной проект разделен на каталоги scripts, scenes, assetss, shaders и blender. В scripts хранится логика, в scenes — структуры сцен, в assetss — импортируемые модели, текстуры и звук, в shaders — код визуального эффекта. Каталог blender содержит исходные файлы трехмерного контента. Такое разделение позволяет отдельно работать над поведением, композицией сцены и ресурсами.

В корневом каталоге также находятся постановка задачи и схемы в формате draw.io XML. Они описывают проектируемые процессы начала игры, радиоперехвата и внекорабельного ремонта. Наличие схем показывает подготовку проектной документации; соответствующие расширенные процессы пока не реализованы полностью в исполняемом коде.

| Файл | Назначение |
|---|---|
| scenes/main.tscn | Основной игровой уровень с игроком, интерьером, светом и радио |
| scenes/player.tscn | Переиспользуемая сцена контроллера персонажа |
| scenes/main_menu.tscn | Главное меню, модель Земли, кнопки и анимация камеры |
| scenes/intro_cutscene.tscn | Вступительное пространство, камера, свет и звуковые узлы |
| scenes/test_scene.tscn | Вспомогательная сцена для экспериментов, требующая доработки |
| scripts/player.gd | Перемещение, обзор, прыжок, фонарь и направленный выбор объекта |
| scripts/interactable.gd | Базовый класс взаимодействующих объектов |
| scripts/radio.gd | Фокус камеры, блокировка игрока и поворот ручки радио |
| scripts/main_menu.gd | Начало игры, скрытие меню, анимация и выход |
| scripts/intro_cutscene.gd | Попытка запуска вступительной анимации и переход после завершения |
| scripts/earthrotating.gd | Непрерывное вращение объекта Земли |
| shaders/dither.gdshader.gdshader | Экранный дизеринг по матрице Байера |
| assetss/default_bus_layout.tres | Настройка главной звуковой шины и фильтра |

## 3 2 Иерархия основной сцены

Корневой узел main имеет тип Node3D. В него добавлен экземпляр Player, узел WorldEnvironment и контейнер Level. В Level находятся импортированный интерьер, источники света и экземпляр модели radio_v03. Узел радио дополнен физическим телом StaticBody3D, формой коллизии и CameraTarget.

Игрок расположен отдельно от геометрии уровня, поэтому его контроллер можно использовать в другом помещении без переписывания основного кода. Свет и статические объекты сгруппированы в Level, что помогает изменять окружение как самостоятельную часть сцены. WorldEnvironment хранит параметры фонового и окружающего освещения.

```
main Node3D
    Player экземпляр scenes/player.tscn
        CollisionShape3D
        Head Node3D
            Camera3D
                RayCast3D
                SpotLight3D
        Node3D
    WorldEnvironment
    Level Node3D
        Без имени экземпляр модели интерьера
        CeilingLamp_Cool
        ConsoleLamp_Amber
        RadioLamp_Cyan
        AirlockEmergency_Red
        PilotNavigationLamp
```

```
radio_v03 экземпляр модели
    импортированные узлы модели
    StaticBody3D со скриптом radio.gd
        CollisionShape3D
    CameraTarget
```



Рисунок 1 — Основная игровая сцена с контрастным освещением

Обзорный кадр основной сцены с отдельной диагностической камеры. Видны организация прохода, геометрия интерьера и зоны голубого и красного освещения.

### 3 3 Связь сцен и программных модулей

В запланированном маршруте новая игра должна проходить через main_menu.tscn, затем intro_cutscene.tscn и main.tscn. Скрипт меню действительно содержит переход к intro_cutscene.tscn после new_animation. Скрипт вступления содержит переход к main.tscn после takeoff. Однако анимации takeoff в сцене вступления нет, поэтому эта часть маршрута сейчас не завершается автоматически.

Внутри основной сцены player.gd определяет попадание луча и вызывает interact. radio.gd наследует Interactable, сохраняет состояние игрока и управляет камерой. Во время фокусировки player.gd передает нажатие D в focus_input объекта радио. Это пример обмена действиями между контроллером персонажа и отдельным игровым объектом через общий интерфейс.

## 4 Трехмерные ресурсы и организация игрового пространства

### 4 1 Состав ресурсов

В каталоге assetss находятся четыре файла GLB: earth.glb, radio_v03.glb, space_shuttle.glb и Без имени.glb. Модель Земли используется в главном меню. Модель радио используется на основном уровне. Модель Без имени.glb задает интерьер основной и вступительной сцен. Файл space_shuttle.glb присутствует среди ресурсов, но в пяти рассматриваемых сценах ссылки на него отсутствуют.

Для материалов имеются 36 внешних изображений PNG и JPG, в том числе две текстуры Земли, три текстуры радио, шесть текстур space_shuttle и 25 изображений с именами Без имени_Image_*. Кроме них, в каталоге blender находится отдельное изображение 1_preview.png. Количество внешних файлов не следует отождествлять с количеством реально используемых текстур в каждом кадре: импортированные модели также содержат данные материалов и могут хранить изображения внутри GLB.

| Модель | Размер файла | Использование |
|---|---|---|
| earth.glb | 15 280 704 байта | Земля в главном меню |
| radio_v03.glb | 5 455 868 байт | Радиоприемник на уровне |
| space_shuttle.glb | 2 738 664 байта | Дополнительный ресурс, не подключенный к указанным сценам |
| Без имени.glb | 18 337 412 байт | Основной интерьер |

Все исходные ресурсы каталога assetss без файлов .import занимают 79 820 568 байт, приблизительно 76,12 МиБ. Эта величина описывает размер файлов ресурсов, а не потребление оперативной памяти и не размер готовой экспортированной игры.

### 4 2 Импорт и размещение интерьера

Модель интерьера добавлена как экземпляр PackedScene. В основной сцене ее положение приблизительно равно (26,655; 0; −5,806), масштаб составляет 1,5 по каждой оси, а ориентация изменена относительно экспортированной модели. Координаты игрока, светильников и радио настроены в той же системе координат.

В исходном GLB интерьера содержится 81 узел, 74 записи мешей и 14 материалов. После импорта в игровую сцену присутствуют физические тела для частей интерьера. Проверка дерева узлов обнаружила StaticBody3D внутри импортированной модели, а физический прогон подтвердил, что персонаж достигает опорной поверхности и получает is_on_floor() == true. Это связывает визуальную геометрию помещения с работой контроллера.

### 4 3 Визуальная композиция помещения

Игровое пространство организовано как ограниченный интерьер корабля. В нем выделяются поверхности стен, проходы, оборудование и локальные источники света. Радиоприемник размещен рядом с голубой подсветкой, которая отделяет его от темного окружения. Теплые и красные источники создают другие зоны внимания и поддерживают образ аварийного состояния.

Важная особенность текущей композиции — высокий контраст. Часть деталей остается в темноте и становится различимой только при включенном фонаре или приближении к источнику света. Это поддерживает атмосферу исследования, но требует проверки читаемости маршрута и расположения интерактивных предметов. Конкретную яркость следует оценивать по кадрам основного уровня при обычной игровой камере.

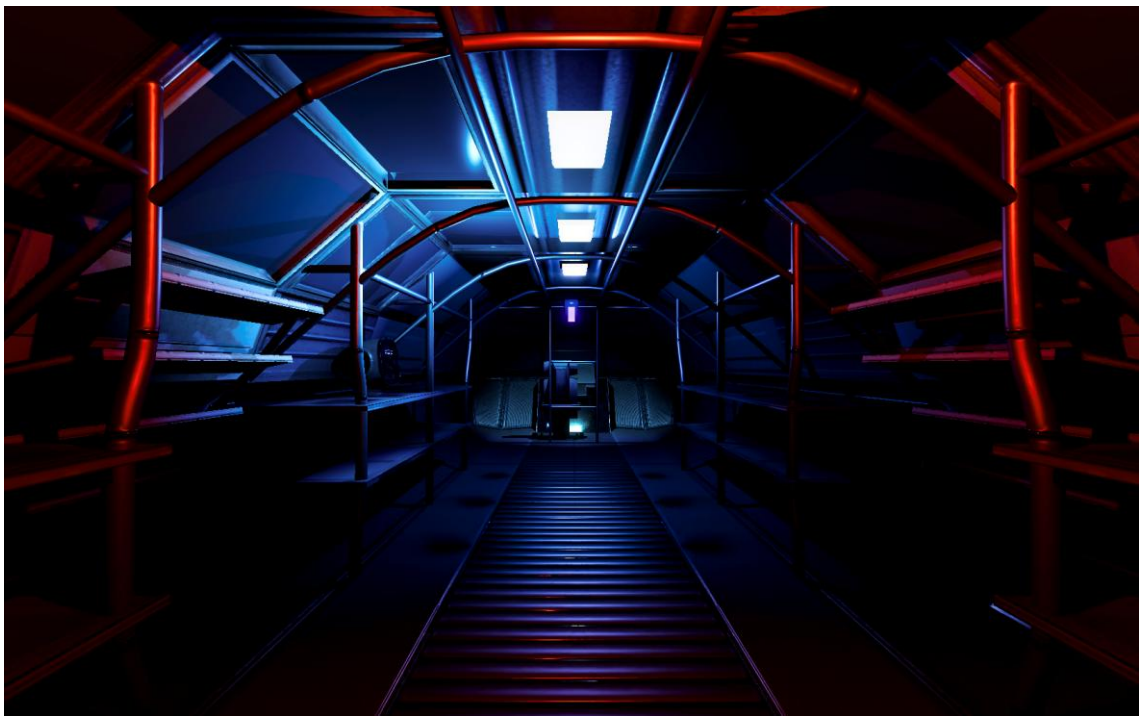


Рисунок 2 — Обратный ракурс интерьера корабля

Второй обзорный кадр той же сцены с диагностической камеры, направленной к противоположной части помещения. Ресурсы и освещение уровня сохранены без изменения.

# 5 Главное меню и переход к новой игре

## 5 1 Состав сцены меню

Главное меню реализовано как трехмерная сцена с двумерными кнопками поверх изображения. В ней находятся модель Земли, Camera3D, DirectionalLight3D, WorldEnvironment, CanvasLayer и AnimationPlayer. Кнопки сгруппированы в VBoxContainer MenuButtons. Состав интерфейса включает «Новая игра», «Продолжить», «Настройки» и «Выйти».

Кнопки визуально присутствуют одновременно, но обработчики подключены только к PlayButton и QuitButton. Узлы Continue и Settings имеют подписи, однако соединений сигнала pressed с программной логикой для них нет. Поэтому в текущем прототипе можно начать переход к вступлению и закрыть приложение; продолжение сохраненной игры и изменение настроек пока являются элементами заготовки интерфейса.

## 5 2 Обработка начала новой игры

Обработчик _on_play_button_pressed сначала скрывает CanvasLayer/MenuButtons, затем запускает new_animation. Эта анимация изменяет позицию камеры за одну секунду. Начальная позиция составляет приблизительно (−0,209; −0,695; −0,515), конечная — (−22,694; 0,192; 1,580). После сигнала animation_finished скрипт проверяет имя завершенной анимации и загружает intro_cutscene.tscn.

Проверка имени нужна потому, что один AnimationPlayer может содержать несколько анимаций. Переход выполняется только для new_animation, а завершение другой анимации не должно автоматически менять сцену. Скрытие кнопок предотвращает их дальнейшее

отображение во время движения камеры, хотя отдельной переменной состояния запуска в скрипте нет.

### 5 3 Выход из приложения

Обработчик кнопки выхода вызывает get_tree().quit(). Это завершает работу SceneTree приложения. Перед выходом не предусмотрены сохранение состояния, запрос подтверждения или очистка отдельно заведенных игровых подсистем, поскольку такие подсистемы в текущей версии отсутствуют.

### 5 4 Анимация Земли

На узел Atmosphere экземпляра earth назначен earthrotating.gd. В _process угол rotation.y уменьшается на 0,2 × delta. Следовательно, угловая скорость равна −0,2 радиана в секунду, приблизительно −11,46 градуса в секунду. Полный оборот при постоянном исполнении занимает около 31,42 секунды. Отрицательный знак задает направление вращения.

Текстура earth_8081_earthmap10k.jpg назначена albedo_texture, а earth_8081_earthspec10k.jpg — roughness_texture материала. Это важно для точного описания реализации: несмотря на название spec в файле, в сцене она используется через параметр шероховатости. Поле зрения камеры меню составляет 31,3 градуса, что задает более узкий обзор модели.

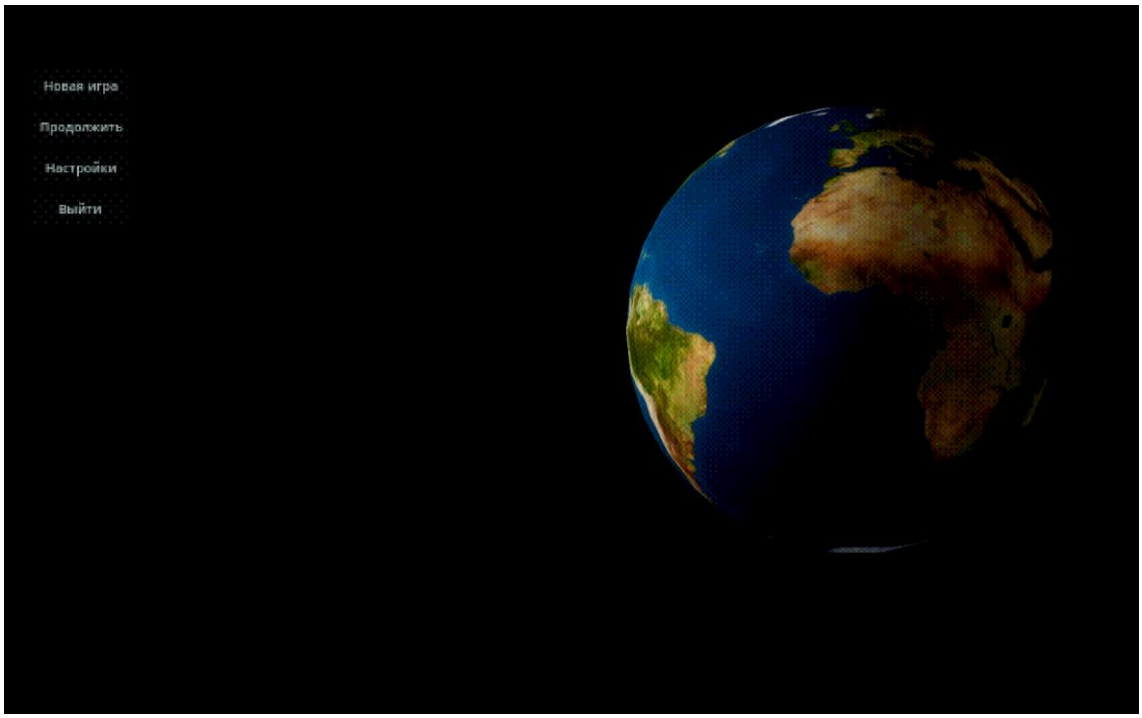



Рисунок 3 — Главное меню с моделью Земли

Фактический кадр main_menu.tscn до запуска новой игры. На экране присутствуют модель Земли и четыре кнопки меню.

# 6 Вступительная сцена

## 6 1 Подготовленное содержимое

В intro_cutscene.tscn размещены импортированный интерьер, камера, пять локальных источников света, экранный ColorRect с материалом дизеринга, AnimationPlayer и два AudioStreamPlayer. Файл EngineSound ссылается на звук взлета космического корабля, а Tick — на запись потрескивания. Геометрия и источники света создают основу вступительного эпизода.

В библиотеке анимаций сцены имеются RESET и new_animation. У new_animation задана длительность 8,366667 секунды. Единственная дорожка относится к позиции Camera3D и содержит два ключа, в которых указана одинаковая позиция. Поэтому даже при запуске этой анимации перемещения камеры по сохраненным ключам не происходит.

## 6 2 Текущая логика запуска

Скрипт intro_cutscene.gd получает ссылку на AnimationPlayer и в _ready вызывает anim_player.play("takeoff"). При завершении анимации с тем же именем выполняется переход в main.tscn. В этой логике оба действия связаны одним идентификатором: именно takeoff должен существовать, запускаться и завершаться.

Фактическая библиотека сцены содержит другое имя. При запуске Godot выводит Animation not found: takeoff. Соответственно, ожидаемая анимация не начинает воспроизведение, а предусмотренное событие завершения не переводит приложение на основной уровень. Это ошибка интеграции сохраненной сцены и скрипта, а не отсутствие самого кода перехода.

## 6 3 Задачи завершения вступления

Для завершения эпизода необходимо согласовать имя анимации в сцене и коде, записать отличающиеся позиции или другие нужные ключи, определить моменты запуска звука и проверить переход после окончания. У AudioStreamPlayer в сохраненной сцене нет autoplay, а в скрипте отсутствуют вызовы play() для EngineSound и Tick. Поэтому подключенные звуковые ресурсы не подтверждают готовое звуковое сопровождение.

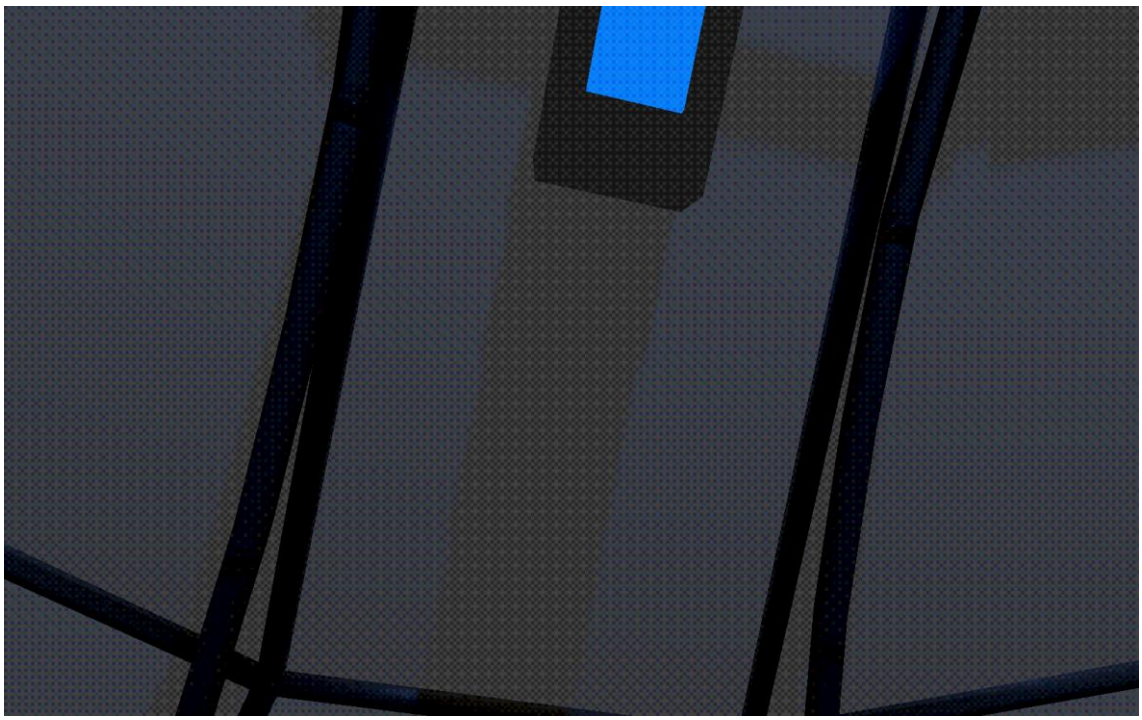


Рисунок 4 — Фактическое изображение вступительной сцены

Кадр intro_cutscene.tscn из сохраненного положения камеры. Камера направлена на близкую геометрию интерьера; виден экранный дизеринг. Ошибка имени анимации, описанная в разделе, не исправлялась при съемке.

# 7 Контроллер персонажа от первого лица

## 7 1 Состав сцены игрока

Сцена player.tscn имеет корневой CharacterBody3D. К нему прикреплен скрипт Player. CollisionShape3D содержит CapsuleShape3D и смещен вверх на 0,9 единицы. Узел Head находится на высоте 1,5 единицы относительно корня персонажа. Camera3D вложена в Head, а RayCast3D и SpotLight3D вложены в камеру.

Такая иерархия позволяет разделить горизонтальный и вертикальный поворот. Head отвечает за вращение вокруг вертикальной оси, а Camera3D — за наклон вверх и вниз. Фонарь и направленный луч следуют за камерой автоматически благодаря родительской связи. В сцене также присутствует дополнительный Node3D, не используемый текущим скриптом.



Рисунок 5 — Вид от первого лица после приземления персонажа

Кадр камеры Player после расчетного приземления на опорную поверхность. Камера направлена вдоль помещения, фонарь выключен; изображение показывает стационарное освещение.

## 7 2 Экспортируемые параметры

В player.gd экспортируются speed, gravity и mouse_sensetivity. Значения по умолчанию в коде составляют 10,0; 9,8 и 0,003 соответственно. Экземпляр Player в main.tscn задает speed = 3.0, gravity = 9.8 и mouse_sensetivity = 0.003. Поэтому на основном уровне используется скорость 3,0, а не исходная скорость 10,0 из объявления переменной.

В самостоятельном файле player.tscn эти три свойства сохранены со значением null. На основном уровне они явно переопределены числовыми значениями. Для корректного повторного использования сцены следует очистить некорректные сохраненные значения и проверить работу player.tscn без переопределений. Это особенно важно при создании новых уровней.

| Параметр | Значение в скрипте | Значение в основном уровне |
|---|---|---|
| speed | 10,0 | 3,0 |
| gravity | 9,8 | 9,8 |
| mouse_sensetivity | 0,003 | 0,003 |
| Скорость прыжка velocity.y | 4,5 | 4,5 |
| Длина направленного луча | Задана в player.tscn | 2,5 единицы |
| Предел наклона камеры | От −89 до +89 градусов | От −89 до +89 градусов |
| Высота Head | Задана в player.tscn | 1,5 единицы |

### 7 3 Захват мыши и обзор

В _ready устанавливается Input.MOUSE_MODE_CAPTURED. Мышь используется для изменения направления взгляда. При событии InputEventMouseMotion горизонтальное смещение event.relative.x поворачивает Head, а вертикальное event.relative.y поворачивает Camera3D. Оба смещения умножаются на mouse_sensetivity; знак минус задает соответствие движения мыши повороту камеры.

После изменения вертикального угла выполняется clamp. Ограничение от −89 до +89 градусов предотвращает полный переворот камеры при продолжительном движении мыши вверх или вниз. Горизонтальный угол не ограничен, поэтому персонаж может осматривать пространство во всех направлениях.

В текущем скрипте нет обработки Escape для освобождения курсора или вызова меню паузы. В режиме радио курсор становится видимым, но обычный игровой режим возвращается к захвату мыши. Отдельная система паузы и интерфейс возврата в меню требуют реализации.

### 7 4 Перемещение клавишами WASD

В каждом физическом кадре создается двухмерный input_dir. Нажатие D увеличивает компоненту x, A уменьшает x, S увеличивает y, W уменьшает y. Если одновременно нажаты противоположные клавиши, их вклад по соответствующей оси взаимно компенсируется.

После чтения клавиш выполняется нормализация input_dir. Без нормализации вектор (1; 1) имел бы длину √2, и движение по диагонали оказалось бы примерно в 1,414 раза быстрее движения вдоль одной оси. Нормализованный вектор имеет единичную длину, поэтому итоговая горизонтальная скорость определяется speed независимо от сочетания направлений.

Далее input_dir преобразуется в трехмерный вектор с нулевой вертикальной компонентой. Умножение на head.global_transform.basis переводит локальное направление в глобальную систему координат. В результате W перемещает персонажа относительно направления головы, а не относительно фиксированной мировой оси.

Если направление ненулевое, velocity.x и velocity.z устанавливаются равными соответствующим компонентам direction × speed. Если клавиши направления не нажаты, обе горизонтальные компоненты становятся нулевыми. Разгон, плавное торможение и инерция для движения внутри помещения сейчас отсутствуют: скорость меняется непосредственно.

### 7 5 Гравитация и прыжок

Если персонаж не находится на полу, velocity.y уменьшается на gravity × delta. При нажатом Space и is_on_floor() == true вертикальная скорость устанавливается в 4,5. После расчета вызывается move_and_slide, который применяет velocity и разрешает столкновения с физическими телами сцены.

Прыжок проверяется через Input.is_key_pressed, то есть через текущее состояние клавиши. Поэтому удержание Space может приводить к новому прыжку после очередного

приземления. Если нужен единичный прыжок на отдельное нажатие, дальнейшая доработка должна заменить эту проверку на событие или на действие с состоянием just_pressed.

При идеализированном движении без потолка и других столкновений начальная скорость 4,5 и ускорение 9,8 дают время подъема около 0,46 секунды и высоту около 1,03 единицы. Это расчетное следствие параметров, а не результат измерения прыжка в конкретной геометрии уровня. Коллизии и физический шаг могут изменить фактическую траекторию.

### 7 6 Блокировка движения при взаимодействии

В начале _physics_process проверяется is_interacting. Если он установлен, velocity становится Vector3.ZERO, вызывается move_and_slide, после чего функция завершается. Чтение WASD, расчет прыжка и обычная гравитация в этой ветке не выполняются. Радио дополнительно устанавливает speed = 0, но основной запрет обычного движения обеспечивается ранним выходом из физической обработки.

Флаг блокирует движение, однако не прекращает всю обработку ввода. В _unhandled_input специальный ранний return существует только для нажатия D в режиме взаимодействия. События E и F могут продолжать обрабатываться. Это соответствует использованию E для выхода и позволяет переключать фонарь у радио, но требует продуманного списка допустимых действий для других приборов.

### 7 7 Назначение клавиш

| Управление | Результат | Условие |
|---|---|---|
| W A S D | Перемещение относительно головы | Обычный режим игрока |
| Движение мыши | Горизонтальный и вертикальный обзор | Курсор захвачен |
| Space | Установка вертикальной скорости прыжка | Персонаж находится на полу |
| E | Попытка взаимодействия с объектом под лучом | Луч попадает в Interactable |
| F | Переключение видимости фонаря | Нажатие клавиши в текущем обработчике |
| D у радио | Поворот ручки на 60 градусов | is_interacting и нажатие без автоповтора |

# 8 Взаимодействие с игровыми объектами

## 8 1 Базовый класс Interactable

Файл interactable.gd объявляет class_name Interactable и наследует Node3D. Он содержит методы interact(_player: CharacterBody3D) и focus_input(_event: InputEvent). Первый метод определяет действие активации объекта, второй предоставляет точку входа для управления после перехода в режим прибора.

Базовая реализация interact выводит отладочное сообщение, а focus_input содержит pass. Это заготовка общего интерфейса: конкретный прибор должен переопределить методы. radio.gd делает это через extends Interactable. Такой подход позволяет подключать новые объекты к существующему лучу без добавления отдельной проверки имени каждого прибора в player.gd.

## 8 2 Выбор объекта лучом

RayCast3D расположен внутри камеры. Его target_position равна (0; 0; −2,5), поэтому луч направлен вперед по локальной оси камеры и ограничен расстоянием 2,5 единицы. При вызове _try_interact контроллер сначала проверяет is_colliding(), затем получает get_collider(). Объект проверяется выражением target is Interactable.

Если проверка типа проходит, ссылка записывается в interacting_object и вызывается target.interact(self). Вызов передает объекту самого игрока, чтобы тот мог получить камеру, скорость и флаг состояния. Если луч ни во что не попал или столкнулся с обычным физическим телом, взаимодействие не запускается.

В модели радио скрипт назначен именно узлу StaticBody3D, по которому должен попадать луч. Базовый Interactable наследуется от Node3D, поэтому такой скрипт может быть использован на производном трехмерном физическом узле. Назначение только визуальной модели без соответствующего физического тела не дало бы нужного результата get_collider().

## 8 3 Ограничения текущей реализации

Направленная проверка не сопровождается экранной подсказкой. Игрок должен знать, что нужно нажать E, и сам направить камеру на объект. Также отсутствуют подсветка выбранного предмета, проверка возможности действия по состоянию системы и текст причины отказа. Эти функции можно добавить поверх существующей логики выбора.

В _try_interact и базовом Interactable сохранены отладочные print с неформальными сообщениями. Они полезны для наблюдения последовательности вызовов во время разработки, но для итоговой версии их следует убрать либо заменить управляемым журналированием. Наличие печати не влияет на само решение о запуске взаимодействия.

# 9 Радиоприемник и режим работы с прибором

## 9 1 Структура интерактивного объекта

Радиоприемник представлен экземпляром radio_v03.glb. Его масштаб в основной сцене равен 0,1 по каждой оси. К модели добавлены StaticBody3D со скриптом radio.gd, CollisionShape3D с BoxShape3D и CameraTarget. Скрипт также обращается к импортированному узлу Radio_TuningKnob_001.

CameraTarget определяет желаемое положение камеры для работы с прибором. Скрипт не переключает активность на эту камеру напрямую: он считывает ее global_transform и перемещает существующую камеру игрока в ту же трансформацию. За счет этого управление одной камерой сохраняется, а возврат можно выполнить к сохраненному положению.

## 9 2 Сохраняемое состояние

В radio.gd объявлены original_camera_transform, original_speed, is_at_radio и is_moving. Первые две переменные сохраняют состояние до начала взаимодействия. is_at_radio показывает, выбран ли сейчас режим работы с прибором. is_moving блокирует повторный запуск взаимодействия, пока предыдущий переход камеры не закончился.

При входе в режим радио original_camera_transform получает текущую глобальную трансформацию камеры игрока. original_speed получает player.speed. Затем speed становится нулевой, is_interacting становится true, target_transform выбирается из CameraTarget, is_at_radio становится true, а курсор переводится в Input.MOUSE_MODE_VISIBLE.

## 9 3 Плавный переход камеры

После выбора целевой трансформации is_moving устанавливается в true. Создается Tween с TRANS_CUBIC и EASE_OUT. tween_property изменяет global_transform камеры до target_transform за 0,5 секунды. Интерполяция включает положение и ориентацию камеры, поскольку передается полная Transform3D.

await tween.finished приостанавливает продолжение метода до окончания перехода. После этого is_moving сбрасывается. Если завершался выход из режима радио, игроку возвращаются original_speed и is_interacting = false, а курсор снова захватывается. Таким образом, обычное движение возобновляется после фактического возвращения камеры.

## 9 4 Выход из режима радио

Повторное interact при is_at_radio == true выбирает original_camera_transform как цель и устанавливает is_at_radio = false. Далее используется тот же механизм Tween. На время обратного перехода игрок остается заблокированным, что предотвращает рассогласование перемещения тела и восстановления камеры.

Контроллер вызывает interact через луч и при повторном нажатии E. Это означает, что выход зависит от текущего попадания луча: если луч больше не видит радио, повторный вызов не состоится. В проверенном положении фокусировки луч попадал в StaticBody3D

радио, и выход по E успешно выполнялся. Для общего решения лучше предусмотреть выход через сохраненную ссылку interacting_object или отдельное действие отмены.

### 9 5 Управление ручкой

В режиме взаимодействия нажатие D без автоповтора передается в focus_input. radio.gd поворачивает Radio_TuningKnob_001 методом rotate_object_local вокруг Vector3.UP на deg_to_rad(60). Одна команда изменяет локальную ориентацию на 60 градусов. Шесть последовательных одинаковых поворотов дают полный оборот вокруг выбранной локальной оси.

Метод focus_input принимает событие, но внутри не проверяет его тип и клавишу. Фильтрация сейчас происходит в player.gd. Поэтому при дальнейшей передаче других событий нужно либо сохранять этот контракт, либо перенести проверку в объект. Переменная knob_angle объявлена, но не участвует в вычислениях; числовая частота также не рассчитывается.

В interact присутствует условие Input.is_key_pressed(KEY_D), внутри которого находится только выражение Radio_TuningKnob. У этого выражения нет команды изменения состояния. Оно не реализует дополнительный поворот или настройку частоты и может быть удалено как незавершенный фрагмент.




Рисунок 6 — Режим работы с радиоприемником

Крупный план после завершения Tween к CameraTarget. В этом режиме скорость игрока равна нулю, а обычное перемещение заблокировано.




Рисунок 7 — Радиоприемник после команды поворота ручки

Кадр того же прибора после команды D, вызывающей поворот Radio_TuningKnob_001 на 60 градусов вокруг локальной оси. Числовое изменение ориентации подтверждено проверкой состояния узла; частота и захват сигнала пока не реализованы.

### 9 6 Результат реализации подсистемы

Текущая подсистема реализует визуальную и управляющую основу физического прибора: близкий ракурс, временную блокировку игрока, отдельный ввод и возврат управления. В ходе проверки скорость изменялась с 3,0 на 0,0 при входе и восстанавливалась до 3,0 при выходе. Флаг is_interacting менялся с false на true и обратно, а ориентация ручки изменялась после команды D.

Для полноценного радиоперехвата нужно добавить диапазон частот, преобразование угла ручки в частоту, целевые сигналы, зависимость шума от отклонения, удержание резонанса, состояние захваченного пакета и связь с терминалом. Эти задачи относятся к следующему функциональному слою и должны использовать уже существующий режим работы с прибором.

## 10 Освещение и управление фонарем

### 10 1 Настройки окружающей среды

В main.tscn фон Environment установлен в почти черный синий цвет Color(0.003, 0.006, 0.014, 1). Цвет окружающего света равен Color(0.025, 0.045, 0.105, 1), энергия — 0,22. Параметр экспозиции tonemap_exposure равен 0,82. Эти настройки задают низкую общую яркость и холодный оттенок темных участков.

В Environment также сохранены glow_enabled, glow_intensity и glow_bloom. Поскольку проект использует GL Compatibility, наличие этих полей в файле не следует считать

подтверждением видимого эффекта свечения. Поддержку и визуальный результат отдельных параметров нужно проверять в используемом способе рендеринга.

### 10 2 Локальные источники света

В основной и вступительной сценах находятся четыре OmniLight3D и один SpotLight3D. Они обозначают холодный потолочный свет, теплую подсветку консоли, голубую зону радио, красное аварийное освещение и направленный навигационный свет. Это статическая расстановка источников; переключения по состоянию аварии текущие скрипты не выполняют.

| Источник | Тип | RGB | Энергия | Дальность |
|---|---|---|---|---|
| CeilingLamp_Cool | OmniLight3D | 0,31; 0,48; 1,00 | 2,4 | 6,0 |
| ConsoleLamp_Amber | OmniLight3D | 1,00; 0,24; 0,055 | 1,55 | 4,5 |
| RadioLamp_Cyan | OmniLight3D | 0,05; 0,55; 0,95 | 7,384 | 2,5 |
| AirlockEmergency_Red | OmniLight3D | 1,00; 0,015; 0,02 | 1,1 | 3,2 |
| PilotNavigationLamp | SpotLight3D | 0,051; 0,549; 0,871 | 10,785 | 6,673 |

Тени явно включены у потолочного, теплого, красного и навигационного источников. У RadioLamp_Cyan поле shadow_enabled в рассматриваемом файле не задано. Некоторые источники также содержат параметры light_indirect_energy. Их присутствие нужно отличать от подтвержденного вклада непрямого освещения в конкретном рендерере.

### 10 3 Фонарь персонажа

Фонарь реализован как SpotLight3D, вложенный в Camera3D. Он следует за взглядом без отдельного вычисления позиции. В player.gd хранится flashlight_flag, а нажатие F меняет этот флаг и visible источника. Две ветки if завершаются return, поэтому за одно событие выполняется только одно переключение.

Начальное значение flashlight_flag равно false. При этом в player.tscn видимость SpotLight3D не выключена, и при проверке его начальное visible оказалось true. Следовательно, первое нажатие F устанавливает true повторно и не выключает уже видимый свет. После второго нажатия состояние становится false и источник скрывается. Для согласованного поведения нужно синхронизировать начальную видимость со значением флага.

Текущие проверки E и F используют event.pressed без исключения event.echo. Поэтому удержание этих клавиш может приводить к повторным попыткам действия при событиях автоповтора. В отличие от них, D в режиме прибора уже содержит проверку not event.echo. Единый подход к дискретным действиям сделает управление предсказуемее.

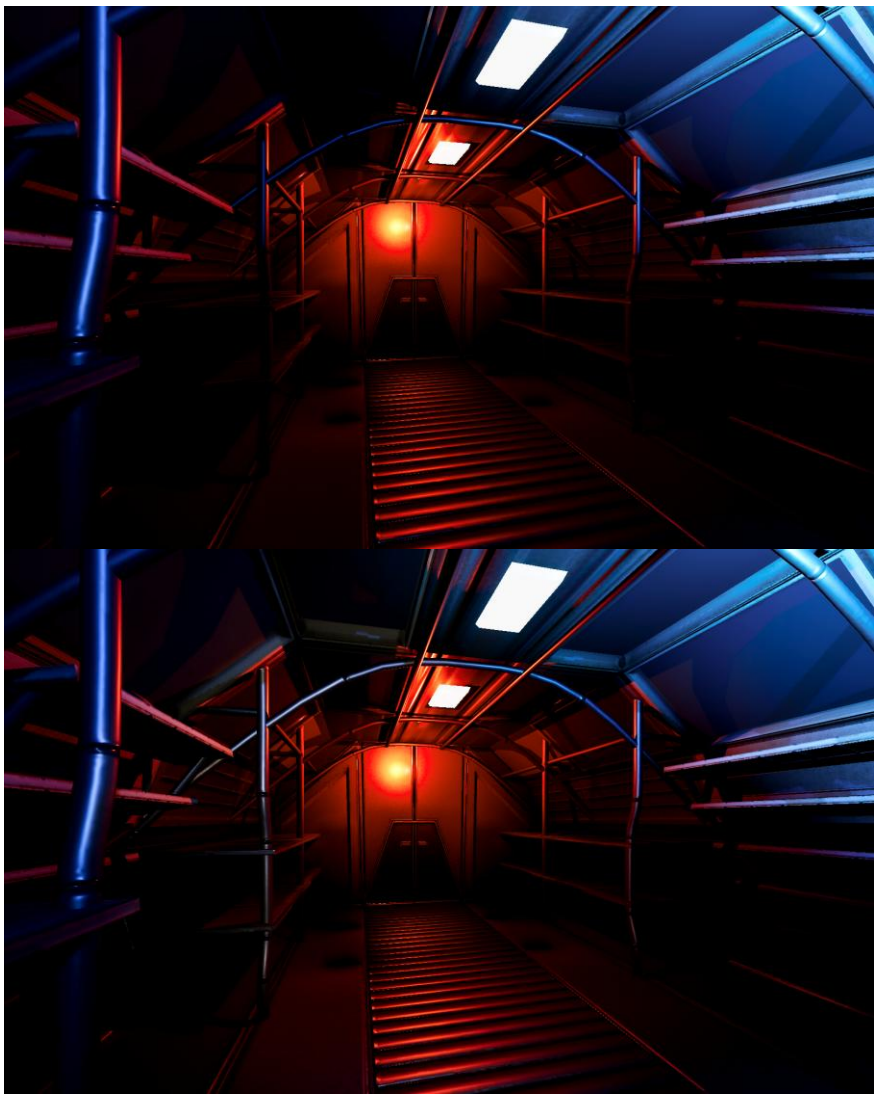


Рисунок 8 — Сравнение одного ракурса с фонарем и без него

Верхний кадр — фонарь выключен; нижний — включен. Положение камеры и стационарные источники одинаковы. Меняется только видимость SpotLight3D персонажа.

# 11 Шейдер дизеринга и визуальный стиль

## 11 1 Назначение эффекта

Файл dither.gdshader.gdshader содержит shader_type canvas_item. Шейдер обрабатывает уже сформированное изображение экрана. SCREEN_TEXTURE объявлена как sampler2D с hint_screen_texture и filter_nearest. В fragment исходный цвет получается через texture(SCREEN_TEXTURE, SCREEN_UV).

Эффект сочетает регулярную матрицу порогов и квантование цвета. Он добавляет к цвету небольшой периодический сдвиг, затем округляет каналы вниз до значений ограниченной палитры. На градиентах и темных поверхностях возникает характерный точечный рисунок. Этот рисунок является экранным эффектом; он не меняет геометрию моделей и физические коллизии.

### 11 2 Матрица Байера

В коде задана матрица 4 × 4 со значениями от 0 до 15, разделенными на 16. Позиция в матрице вычисляется как ivec2(FRAGCOORD.xy / dither_size) % 4. Деление на dither_size увеличивает размер элемента сетки, а остаток от деления на четыре делает узор периодическим по обеим осям.

Из выбранного значения вычитается 0,5. Полученный dither_value делится на color_depth и добавляется ко всем трем каналам исходного цвета. Затем floor(color × color_depth) / color_depth ограничивает ступени яркости. Альфа-канал берется из screen_color.a без дополнительного квантования.

### 11 3 Параметры и подключение

В коде dither_size имеет диапазон 1–8 и значение по умолчанию 2. color_depth имеет диапазон 2–64 и значение по умолчанию 16. В материале главного меню установлены dither_size = 4 и color_depth = 12. В сцене вступления заданы dither_size = 5 и color_depth = 12.

В обеих сценах материал назначен полноэкранному ColorRect в CanvasLayer. Его якоря охватывают всю область, а mouse_filter = 2 позволяет не перехватывать указатель над расположенными под ним кнопками. В main.tscn такого ColorRect и ссылки на шейдер нет. Поэтому ретроэффект подключен к меню и вступлению, но не ко всему игровому проекту автоматически.

Меньшая color_depth увеличивает заметность ступеней и силу изменения цвета. Большая dither_size делает рисунок крупнее. Значение color_depth не обозначает число бит изображения или готовую палитру из фиксированных цветов: это параметр шага квантования каналов в конкретной формуле.

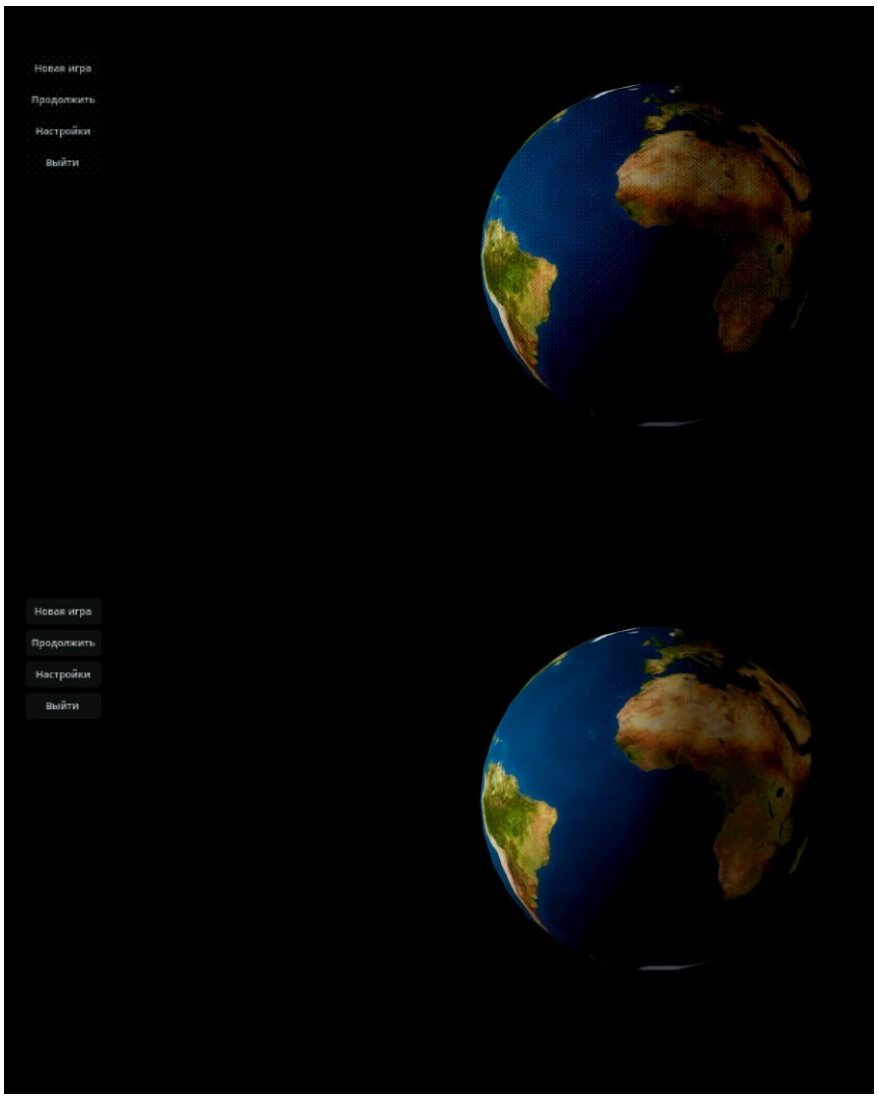



Рисунок 9 — Сравнение изображения меню с дизерингом и без него

Верхний кадр — ColorRect с шейдером включен, dither_size = 4 и color_depth = 12; нижний — этот экранный слой скрыт. Положение камеры и угол Земли одинаковы, чтобы показать действие эффекта.

# 12 Звуковые ресурсы

## 12 1 Подготовленные звуки

В проект добавлены два MP3: the-sound-of-a-spaceship-taking-off-a-rocket.mp3 и vinilovoe-tploe-poskripyivanie-ts100065.mp3. Они подключены к EngineSound и Tick во вступительной сцене. EngineSound имеет volume_db = 3.902. Скрипт вступления не запускает эти проигрыватели, а анимация не содержит звуковых дорожек.

Текущий результат работы со звуком состоит в подключении ресурсов и подготовке узлов воспроизведения. Во время диагностического запуска значение EngineSound.playing было false. Звуковые файлы еще необходимо синхронизировать с действиями сцены и проверить обычное слышимое воспроизведение.

### 12 2 Звуковая шина

В project.godot default_bus_layout ссылается на default_bus_layout.tres. На главной шине указана громкость −2,312501 дБ. Добавлен и включен эффект AudioEffectFilter. Отдельные шины для радио, окружения, интерфейса или звука внутри шлема в текущем файле не описаны.

Для последующего развития проекта потребуется связать звук с состояниями игры. Радиосистема должна изменять шум в зависимости от частоты, а режим открытого космоса — переключать звуковое окружение. В текущих программных модулях таких зависимостей нет, поэтому они остаются требованиями дальнейшей реализации.

## 13 Проверка проекта и результаты испытаний

### 13 1 Условия проверки

Проверка выполнена 1 октября 2026 года в Windows с Godot Engine 4.7.2 stable. При отображении сцен использовался GL Compatibility, OpenGL 3.3 и адаптер Intel Iris Xe Graphics. Диагностическая область отображения соответствовала 1680 × 1050. Для служебного прогона был выбран звуковой драйвер Dummy, поэтому результат не подтверждает качество слышимого звука.

Проверка включала чтение конфигурации и сцен, загрузку проекта редактором без отображения интерфейса, короткий запуск основной сцены и отдельный диагностический прогон существующих игровых модулей. Для отдельных действий использовались программно созданные события клавиатуры и прямой вызов методов. Это позволяет подтвердить изменения состояний, но не заменяет полный ручной проход всех игровых ситуаций.

Диагностическая обработка физики выполнялась 120 шагов по 1/60 секунды. Для сценового показа персонаж временно останавливался в служебном экземпляре; игровые исходники при этом не изменялись. Для проверки направленного взаимодействия дополнительно выбирались положение игрока рядом с радио и направление камеры на его форму коллизии.

### 13 2 Результаты выполненных проверок

| Проверка | Полученный результат | Вывод |
|---|---|---|
| Загрузка project.godot редактором | Проект загружен; в конце журнала два сообщения get_global_transform вне дерева | Требуется наблюдение за редакторской диагностикой |
| Короткий запуск главной сцены | main.tscn запускается; ошибок скрипта в журнале короткого запуска нет | Базовый старт уровня подтвержден |

| Проверка | Полученный результат | Вывод |
|---|---|---|
| Наличие анимации меню | new_animation найдена | Скрипт меню ссылается на существующую анимацию |
| Наличие вступительной анимации | takeoff не найдена; имеются RESET и new_animation | Автоматический маршрут через вступление нарушен |
| Начальное состояние фонаря | visible = true при flashlight_flag = false | Начальные состояния рассогласованы |
| Первое и второе действие F | Видимость стала true, затем false | Ветви переключения исполняются; начальное состояние требует исправления |
| Физика персонажа после 120 шагов | is_on_floor() = true | Коллизия с опорной поверхностью подтверждена |
| E рядом с радио при направленном луче | Луч попал в тело радио, is_interacting = true | Выбор Interactable через луч подтвержден |
| Вход в режим радио | speed = 0, is_interacting = true | Блокировка обычного движения подтверждена |
| Команда D в режиме радио | Локальная ориентация ручки изменилась | Вызов focus_input и поворот подтверждены |
| Выход по E из проверенного ракурса | is_interacting = false, speed = 3 | Возврат управления подтвержден для проверенного положения |
| Состояние EngineSound при запуске вступления | playing = false | Автоматическое воспроизведение не настроено |

### 13 3 Проверка движения и коллизий

Начальное положение персонажа в основной сцене составляет приблизительно (26,122; 3,616; −5,495). После двух расчетных секунд физики персонаж оказался около (25,637; 2,021; −5,528) и находился на полу. Вертикальное смещение соответствует падению к опорной поверхности. Небольшое изменение горизонтальных координат связано с разрешением столкновений в стартовом размещении.

Следовательно, начальная позиция персонажа выше положения устойчивой опоры. Для итоговой версии желательно установить точку появления так, чтобы камера сразу давала понятный ракурс, а физическое тело не требовало заметного начального перемещения. Проверка одной точки не подтверждает отсутствие застреваний или прохода сквозь геометрию во всем помещении.

### 13 4 Проверка взаимодействия

Первоначальная попытка выбора радио из стартового положения не дала попадания луча. При размещении персонажа рядом с прибором, в диагностической позиции (27,3; 2,2; −5,7), и

направлении камеры на форму коллизии луч обнаружил именно тело радио. Нажатие E через обработчик игрока включило режим взаимодействия.

Дополнительно механизм фокусировки проверялся прямым вызовом interact. Такой вызов подтверждает работу объекта и Tween отдельно от дальности луча. После входа ориентация ручки изменилась при D, а повторное E в проверенном фокусном положении вернуло обычный режим. Для общего пользовательского сценария необходимо проверить несколько дистанций, углов подхода и попыток выхода во время перехода.

### 13 5 Проверки по исходному коду

По коду подтверждены нормализация направления движения, ограничение вертикального угла камеры, условие прыжка на полу, проверка типа Interactable и защита радио через is_moving. Проверка алгоритма не равнозначна количественному испытанию всех его свойств. В рамках текущего отчета не измерялись диагональная скорость, высота прыжка и длительность перехода внешним таймером.

Для кнопок «Продолжить» и «Настройки» установлено отсутствие обработчиков pressed. У вспомогательной test_scene.tscn не хватает Head, Camera3D, RayCast3D и SpotLight3D, на которые ссылается player.gd. Поэтому ее нельзя считать завершенным стендом проверки текущего контроллера без дополнения иерархии или замены игрока экземпляром player.tscn.

### 13 6 Оставшиеся испытания

Полный ручной проход должен охватить старт через главное меню, повторное начало игры, движение возле всех стен и дверных проемов, прыжки под низкими поверхностями, разные ракурсы входа в радио и выход во время поворота ручки. После исправления вступления необходимо подтвердить воспроизведение звуков и однократную загрузку основного уровня.

Требование 60 кадров в секунду при 1920 × 1080 пока не подтверждено измерениями. Короткий диагностический запуск не является нагрузочным тестом. Для проверки производительности нужно выбрать фиксированные маршруты камеры, записать время кадра на целевых устройствах и отдельно оценить влияние теней, материалов, количества источников света и экранного шейдера.

## 14 Соответствие постановке задачи

### 14 1 Сопоставление функций

Исходная постановка задачи описывает полный игровой продукт, а текущий код реализует его базовый прототип. Таблица показывает конкретную степень готовности каждого направления. Статус «частично» означает, что подготовлены отдельные элементы, но требуемый пользовательский сценарий еще не завершается полностью.

| Требование | Состояние | Основание |
|---|---|---|
| Трехмерная игра на Godot с GDScript | Реализована основа | Проект, сцены, типизированные скрипты |

| Требование | Состояние | Основание |
|---|---|---|
| Использование Jolt Physics | Настроено | Параметр 3d/physics_engine |
| Главное меню | Частично | Кнопки есть; обработаны начало и выход |
| Продолжение игры и настройки | Не реализованы | Нет обработчиков и системы сохранений |
| Плавный переход перед вступлением | Реализована основа | new_animation в меню и обработчик завершения |
| Вступление и переход на уровень | Частично | Сцена есть; имя анимации не согласовано |
| Меню паузы | Не реализовано | Нет соответствующих узлов и обработчиков |
| WASD и обзор мышью | Реализованы | player.gd и иерархия Head с Camera3D |
| Прыжок и гравитация | Реализована основа | Расчет velocity и move_and_slide |
| Фонарь | Реализована основа | SpotLight3D и F; начальная видимость требует исправления |
| Направленное взаимодействие | Реализовано для радио | RayCast3D и Interactable |
| Фокусировка на приборе и возврат | Реализованы | Transform3D, Tween и восстановление speed |
| Диапазон 100,00–999,99 МГц | Не реализован | В radio.gd отсутствует модель частоты |
| Шум и осциллограмма от частоты | Не реализованы | Нет вычислений и интерфейса сигнала |
| Удержание резонанса три секунды | Не реализовано | Нет таймера и целевого сигнала |
| Радиопакет и расшифровка | Не реализованы | Нет структуры пакета и терминала |
| Система неисправностей | Не реализована | Есть только визуальная аварийная подсветка |
| Безопасное шлюзование | Не реализовано | Нет состояния люков и давления |
| Выход в открытый космос и захваты | Не реализованы | Контроллер работает в обычной гравитации |
| Кислород и шлемовый интерфейс | Не реализованы | Нет соответствующих данных и HUD |
| Ремонт у корпуса | Не реализован | Нет состояния фиксации и ремонтных точек |
| Ретродизеринг | Частично | Подключен в меню и вступлении |
| Звуковая атмосфера | Частично | Ресурсы подключены, запуск звука не настроен |

| Требование | Состояние | Основание |
|---|---|---|
| JSON сохранение прогресса | Не реализовано | Нет чтения и записи состояния игры |
| Контекстные подсказки и журнал сигналов | Не реализованы | Нет соответствующих интерфейсов |
| 60 FPS при 1920 × 1080 | Не подтверждено | Нагрузочные измерения не выполнялись |

### 14 2 Что подтверждает выполненную работу

Реализацию базового управления подтверждают player.gd и player.tscn. Организацию уровня подтверждает main.tscn с экземпляром игрока, импортированным интерьером, светом и радио. Взаимодействие подтверждается общим классом Interactable и выполненными проверками перехода в режим радио. Визуальную стилизацию подтверждают код шейдера и материалы двух сцен.

Для функций, отмеченных как не реализованные, в текущем проекте отсутствует соответствующая исполняемая логика. Схемы радиоперехвата и ремонта описывают проектируемые процессы и могут использоваться при дальнейшей разработке. Их следует рассматривать как проектную основу, а не как доказательство завершенных игровых механик.

## 15 Обнаруженные ограничения и направления доработки

### 15 1 Завершение запуска приложения

Первым шагом следует назначить main_menu.tscn главной сценой и изменить config/name на ECHOES OF THE VOID. Затем необходимо согласовать название вступительной анимации в intro_cutscene.gd и AnimationLibrary. После исправления требуется проверить маршрут «Новая игра — анимация меню — вступление — основной уровень» целиком.

Для самого вступления нужно задать содержательные ключи камеры или объектов, настроить запуск звука и обеспечить понятный первый кадр. Сохраненные ключи с одинаковой позицией не создают движение. Отдельно нужно определить, должна ли сцена пропускаться пользователем, и добавить соответствующее управление, если оно требуется дизайном игры.

### 15 2 Согласование состояния игрока

Следует исправить null у экспортируемых свойств в player.tscn, привести начальную видимость фонаря к flashlight_flag и разместить персонажа в устойчивой точке появления. Для E и F рекомендуется исключить event.echo, чтобы отдельное действие не повторялось при удержании. Для прыжка нужно определить, допустим ли повторный прыжок сразу после приземления при удержанном Space.

Для развития ввода полезно завести именованные действия InputMap: движение, взаимодействие, фонарь, прыжок, отмена и пауза. Текущая версия использует прямые

KEY_*, поэтому назначение клавиш и поддержка альтернативных устройств связаны с изменением кода. Переход к действиям можно выполнить без изменения основных физических расчетов.

### 15 3 Устойчивость взаимодействия

Выход из режима прибора следует сделать доступным через сохраненный interacting_object, независимо от текущего попадания луча. В момент входа и выхода полезно явно управлять разрешенными действиями и состоянием камеры. Также необходимо проверять существование узлов CameraTarget и ручки, чтобы новая модель прибора не приводила к ошибке из-за отличающегося имени дочернего узла.

На следующем этапе нужно добавить контекстную подсказку E, визуальное выделение доступного предмета и понятное действие завершения работы. Общий Interactable можно расширить названием действия и проверкой can_interact, если приборы будут зависеть от питания или состояния аварии. Это развитие существующего интерфейса, а не готовая функция текущей версии.

### 15 4 Полноценная радиосистема

Радиосистема должна хранить текущую частоту и допустимый диапазон. Поворот ручки должен изменять частоту по выбранному шагу либо по нормализованному углу. Для каждого сигнала нужно определить целевую частоту, допустимое отклонение и длительность удержания. Уровень шума и вид осциллограммы должны изменяться по вычисляемой близости к целевой частоте.

После успешного удержания требуется сформировать радиопакет с идентификатором, состоянием захвата, данными для расшифровки и ссылкой на звук. Терминал должен показывать доступные пакеты, выполнять этап расшифровки и воспроизводить результат. Состояния нужно связать с журналом сигналов и сохранением, чтобы прогресс не терялся между запусками.

### 15 5 Системы корабля и выход в космос

Для шлюзования нужна отдельная модель состояний внутреннего люка, внешнего люка и давления. Открытие внешнего люка должно быть разрешено только при закрытом внутреннем и завершенной декомпрессии. Переход в космос должен менять физическое управление и звуковое окружение, а не только отключать гравитацию.

Контроллер внекорабельной деятельности должен учитывать инерцию, захват поверхности и ограничение ремонта устойчивой фиксацией. Кислород необходимо хранить в игровом состоянии и отображать на интерфейсе. Система аварий должна задавать конкретные неисправности, условия устранения и изменения сигнализации. Эти подсистемы существенно шире текущего контроллера помещений.

### 15 6 Сохранение и качество сборки

Для JSON сохранения следует описать структуру состояния: текущая сцена, положение игрока, запас кислорода, состояния корабля и найденные сигналы. При загрузке необходимо

проверять версию данных и корректность значений. После этого можно связать кнопку «Продолжить» с существующим сохранением и определить поведение, когда сохранения нет.

Финальный этап качества должен включать исправление вспомогательной test_scene.tscn, удаление неиспользуемых переменных и отладочных сообщений, проверку полного маршрута игры и измерение производительности. Возможность запуска проекта в редакторе пока не означает готовность экспортированной сборки для Windows 10 и Windows 11. Экспорт и проверка на целевых системах являются отдельными критериями завершения.

## 16 Итоги выполненной работы

В ходе текущего этапа создана основа трехмерного игрового проекта ECHOES OF THE VOID. Организованы файлы сцен и скриптов, подключены модели и текстуры, сформирован интерьер с локальным освещением. Написан контроллер персонажа с движением относительно направления взгляда, гравитацией, прыжком, направленным лучом и управляемым фонарем.

Разработан общий интерфейс Interactable и прототип физического радиоприемника. Реализованы сохранение положения камеры, плавный переход к прибору, блокировка движения, поворот ручки и возврат управления. Подготовлены главное меню с вращающейся Землей, сцена вступления и экранный шейдер дизеринга. Звуковые ресурсы подключены к узлам вступительной сцены.

Проверка подтвердила запуск основного уровня, наличие коллизий интерьера и работу ключевых состояний радио. Одновременно выявлены несогласованное имя вступительной анимации, прямой запуск уровня вместо меню, начальное рассогласование фонаря и незавершенные элементы интерфейса. Устранение этих ограничений позволит завершить базовый пользовательский маршрут, после чего проект можно расширять радиоперехватом, системами корабля и внекорабельным ремонтом.

# Приложение А Полные листинги программных модулей

Листинги воспроизводят текущие файлы проекта без исправления программной логики. Слева добавлены номера исходных строк для сопоставления с пояснениями. Пустые строки и комментарии сохранены. Отладочные сообщения и незавершенные фрагменты также приведены в исходном виде.

## А 1 Контроллер игрока

Файл scripts/player.gd. Строки 1–14 задают тип, состояния, экспортируемые параметры и ссылки на узлы. Строки 17–18 включают захват мыши. Строки 20–29 выбирают объект через RayCast3D. Строки 31–55 обрабатывают мышь, радио и фонарь. Физическая логика находится в строках 64–94 и завершается move_and_slide.

```
001  class_name Player
002  extends CharacterBody3D
003
004  var flashlight_flag: bool = false
005  var interacting_object: Interactable = null
006
007  @export var speed: float = 10.0
008  @export var gravity: float = 9.8
009  @export var mouse_sensetivity: float = 0.003
010
011  @onready var head: Node3D = $Head
012  @onready var camera: Camera3D = $Head/Camera3D
013  @onready var interaction_ray: RayCast3D = $Head/Camera3D/RayCast3D
014  @onready var flashlight: SpotLight3D = $Head/Camera3D/SpotLight3D
015
016
017  func _ready() -> void:
018      Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
019
020  func _try_interact() -> void:
021      print("Я нажал E")
022      if interaction_ray.is_colliding():
023          print("Луч что то нашел")
024          var target = interaction_ray.get_collider()
025          print(target)
026          print("Это Interactable", target is Interactable)
027          if target is Interactable:
028              interacting_object = target
029              target.interact(self)
030
031  func _unhandled_input(event: InputEvent) -> void:
032      if is_interacting:
033          if event is InputEventKey and event.pressed and not event.echo and event.keycode == KEY_D:
034              interacting_object.focus_input(event)
035              return
036
037      if event is InputEventMouseMotion and Input.mouse_mode == Input.MOUSE_MODE_CAPTURED:
038          head.rotate_y(-event.relative.x * mouse_sensetivity)
039
040          camera.rotate_x(-event.relative.y * mouse_sensetivity)
041
042          camera.rotation.x = clamp(camera.rotation.x, deg_to_rad(-89.0), deg_to_rad(89.0))
043
044      if event is InputEventKey and event.pressed and event.keycode == KEY_E:
045          _try_interact()
046
047      if event is InputEventKey and event.pressed and event.keycode == KEY_F:
048          if flashlight_flag == false:
049              flashlight_flag = true
050              flashlight.visible = true
051              return
052          if flashlight_flag == true:
053              flashlight_flag = false
054              flashlight.visible = false
055              return
056
```

```
057
058  var is_interacting: bool = false
059
060  func _unhandled_key_input(event: InputEvent) -> void:
061      if is_interacting:
062          return
063
064
065  func _physics_process(delta: float) -> void:
066      if is_interacting:
067          velocity = Vector3.ZERO
068          move_and_slide()
069          return
070
071      var input_dir: Vector2 = Vector2.ZERO
072
073      if is_on_floor() and Input.is_key_pressed(KEY_SPACE):
074          velocity.y = 4.5
075
076      if not is_on_floor():
077          velocity.y -= gravity * delta
078
079      if Input.is_key_pressed(KEY_D): input_dir.x += 1
080      if Input.is_key_pressed(KEY_A): input_dir.x -= 1
081      if Input.is_key_pressed(KEY_S): input_dir.y += 1
082      if Input.is_key_pressed(KEY_W): input_dir.y -= 1
083      input_dir = input_dir.normalized()
084
085      var direction: Vector3 = (head.global_transform.basis * Vector3(input_dir.x, 0,
input_dir.y)).normalized()
086
087      if direction != Vector3.ZERO:
088          velocity.x = direction.x * speed
089          velocity.z = direction.z * speed
090      else:
091          velocity.x = 0.0
092          velocity.z = 0.0
093
094      move_and_slide()
```

## А 2 Базовый класс взаимодействующих объектов

Файл scripts/interactable.gd. Класс задает общий контракт interact и focus_input. Объекты, наследующие Interactable, переопределяют эти методы для своего поведения.

```
001  class_name Interactable
002  extends Node3D
003
004  func interact(_player: CharacterBody3D) -> void:
005      print("ДОООООООО")
006
007  func focus_input(_event: InputEvent) -> void:
008      pass
```

## А 3 Работа с радиоприемником

Файл scripts/radio.gd. Объявления состояния находятся в начале файла. focus_input поворачивает ручку, а interact выбирает направление перехода камеры. В конце файла ожидание завершения Tween связано с возвратом скорости, флага взаимодействия и захвата мыши.

```
001  extends Interactable
002
003  @onready var camera_target: Camera3D = $"../CameraTarget"
004  @onready var Radio_TuningKnob: Node3D = $"../Radio_TuningKnob_001"
005
006  var knob_angle: float = 0.0
007  var original_camera_transform: Transform3D
```

```
var original_speed: float
var is_at_radio := false
var is_moving := false


func focus_input(event: InputEvent) -> void:
    Radio_TuningKnob.rotate_object_local(Vector3.UP,deg_to_rad(60))



func interact(player: CharacterBody3D) -> void:
    if is_moving:
        return

    var player_camera: Camera3D = player.get_node("Head/Camera3D")

    var target_transform: Transform3D

    if not is_at_radio:
        # Сохраняем положение камеры и скорость до перемещения
        original_camera_transform = player_camera.global_transform
        original_speed = player.speed

        player.speed = 0
        player.is_interacting = true

        target_transform = camera_target.global_transform
        is_at_radio = true

        Input.mouse_mode = Input.MOUSE_MODE_VISIBLE

        if Input.is_key_pressed(KEY_D):
            Radio_TuningKnob


    else:
        # Возвращаемся ровно туда, где камера была раньше
        target_transform = original_camera_transform
        is_at_radio = false

    is_moving = true

    var tween := create_tween()
    tween.set_trans(Tween.TRANS_CUBIC)
    tween.set_ease(Tween.EASE_OUT)

    tween.tween_property(
        player_camera,
        "global_transform",
        target_transform,
        0.5
    )

    await tween.finished

    is_moving = false

    if not is_at_radio:
        player.speed = original_speed
        player.is_interacting = false

        Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
```

## А 4 Управление главным меню

Файл scripts/main_menu.gd. Обработчик выхода закрывает SceneTree, обработчик новой игры скрывает кнопки и запускает анимацию. Сигнал завершения new_animation переводит приложение во вступительную сцену.

```
extends Node3D

@onready var anim_player: AnimationPlayer = $AnimationPlayer

```

```
005  func _on_quit_button_pressed() -> void:
006      get_tree().quit()
007
008
009  func _on_play_button_pressed() -> void:
010      $CanvasLayer/MenuButtons.hide()
011      anim_player.play("new_animation")
012
013  func _on_animation_player_animation_finished(anim_name: StringName) -> void:
014      if anim_name == "new_animation":
015          get_tree().change_scene_to_file("res://scenes/intro_cutscene.tscn")
```

## А 5 Вступительная сцена

Файл scripts/intro_cutscene.gd. Идентификатор takeoff используется при запуске и проверке окончания. В сохраненной сцене это имя отсутствует, что объясняет обнаруженную ошибку перехода.

```
001  extends Node3D
002  @onready var anim_player: AnimationPlayer = $AnimationPlayer
003
004  func _ready() -> void:
005      anim_player.play("takeoff")
006
007  func _on_animation_player_animation_finished(anim_name: StringName) -> void:
008      if anim_name == "takeoff":
009          get_tree().change_scene_to_file("res://scenes/main.tscn")
```

## А 6 Вращение Земли

Файл scripts/earthrotating.gd. Активная строка меняет rotation.y пропорционально delta. Закомментированное вращение вокруг x не выполняется.

```
001  extends MeshInstance3D
002
003  func _process(delta: float) -> void:
004      #rotation.x += 0.2 * delta
005      rotation.y += -0.2 * delta
```

## А 7 Экранный шейдер дизеринга

Файл shaders/dither.gdshader.gdshader. Листинг включает объявление экранной текстуры, параметры размера сетки и квантования, матрицу Байера и функцию fragment. Этот файл подключен к материалам меню и вступления.

```
001  shader_type canvas_item;
002
003  // Захватываем картинку 3D-экрана
004  uniform sampler2D SCREEN_TEXTURE : hint_screen_texture, filter_nearest;
005
006  // Настройки размера точек и палитры в Инспекторе
007  uniform float dither_size : hint_range(1.0, 8.0, 1.0) = 2.0;
008  uniform float color_depth : hint_range(2.0, 64.0, 1.0) = 16.0;
009
010  // Матрица дизеринга 4x4 (Матрица Байера)
011  const mat4 bayer_matrix = mat4(
012      vec4(0.0, 8.0, 2.0, 10.0),
013      vec4(12.0, 4.0, 14.0, 6.0),
014      vec4(3.0, 11.0, 1.0, 9.0),
015      vec4(15.0, 7.0, 13.0, 5.0)
016  ) / 16.0;
017
018  void fragment() {
019      // 1. Берем исходный цвет пикселя 3D-мира
```

```
020      vec4 screen_color = texture(SCREEN_TEXTURE, SCREEN_UV);
021
022      // 2. Вычисляем пиксельную позицию сетки
023      ivec2 screen_pos = ivec2(FRAGCOORD.xy / dither_size) % 4;
024      float dither_value = bayer_matrix[screen_pos.x][screen_pos.y] - 0.5;
025
026      // 3. Подмешиваем точечную сетку к цвету
027      vec3 color = screen_color.rgb + vec3(dither_value / color_depth);
028
029      // 4. Ограничиваем количество цветов (квантование палитры)
030      color = floor(color * color_depth) / color_depth;
031
032      COLOR = vec4(color, screen_color.a);
033  }
```

# Приложение Б Полные листинги сцен и конфигурации

Текстовые сцены приведены полностью, поскольку они определяют фактическое размещение объектов, переопределения параметров игрока, имена анимаций и соединения сигналов. Длинные строки ресурсов и Transform3D могут переноситься на следующую визуальную строку; номер относится к исходной строке файла. UID, имена ресурсов и параметры сохранены без изменения.

## Б 1 Конфигурация проекта

Файл project.godot. В нем указаны стартовая сцена, имя приложения, базовое разрешение, звуковая шина, Jolt Physics и способ визуализации.

```
001  ; Engine configuration file.
002  ; It's best edited using the editor UI and not directly,
003  ; since the parameters that go here are not all obvious.
004  ;
005  ; Format:
006  ;   [section] ; section goes between []
007  ;   param=value ; assign values to parameters
008
009  config_version=5
010
011  [application]
012
013  config/name="test"
014  run/main_scene="uid://dqqbblk6vnnqr"
015  config/features=PackedStringArray("4.7", "GL Compatibility")
016  config/icon="res://icon.svg"
017
018  [audio]
019
020  buses/default_bus_layout="uid://doeotxymw54lm"
021
022  [display]
023
024  window/size/viewport_width=1680
025  window/size/viewport_height=1050
026  window/stretch/mode="canvas_items"
027
028  [filesystem]
029
030  import/blender/enabled=false
031
032  [physics]
033
034  3d/physics_engine="Jolt Physics"
035
036  [rendering]
037
038  textures/canvas_textures/default_texture_filter=0
039  rendering_device/driver.windows="d3d12"
040  renderer/rendering_method="gl_compatibility"
041  renderer/rendering_method.mobile="gl_compatibility"
```

## Б 2 Сцена игрока

Файл scenes/player.tscn. Он задает форму персонажа, высоту головы, вложенность камеры, длину луча и узел фонаря. Значения null у экспортируемых полей показаны в том виде, в котором они сохранены.

```
001  [gd_scene format=3 uid="uid://cgjj2bmatef67"]
002
003  [ext_resource type="Script" uid="uid://d3cpik1ayfhsl" path="res://scripts/player.gd" id="1_3vyb7"]
004
005  [sub_resource type="CapsuleShape3D" id="CapsuleShape3D_rrmyu"]
```

```
006
007  [node name="Player" type="CharacterBody3D" unique_id=502170894]
008  script = ExtResource("1_3vyb7")
009  speed = null
010  gravity = null
011  mouse_sensetivity = null
012
013  [node name="CollisionShape3D" type="CollisionShape3D" parent="." unique_id=1017400892]
014  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0.9, 0)
015  shape = SubResource("CapsuleShape3D_rrmyu")
016
017  [node name="Head" type="Node3D" parent="." unique_id=64070310]
018  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1.5, 0)
019
020  [node name="Camera3D" type="Camera3D" parent="Head" unique_id=1770491588]
021
022  [node name="RayCast3D" type="RayCast3D" parent="Head/Camera3D" unique_id=823453081]
023  target_position = Vector3(0, 0, -2.5)
024
025  [node name="SpotLight3D" type="SpotLight3D" parent="Head/Camera3D" unique_id=1967316689]
026
027  [node name="Node3D" type="Node3D" parent="." unique_id=1005335123]
```

## Б 3 Основной игровой уровень

Файл scenes/main.tscn. Здесь заданы числовые параметры экземпляра Player, Environment, расстановка света, модели интерьера и радио, форма взаимодействия и CameraTarget.

```
001  [gd_scene format=3 uid="uid://dqqbblk6vnnqr"]
002
003  [ext_resource type="PackedScene" uid="uid://cgjj2bmatef67" path="res://scenes/player.tscn"
id="1_player_scene"]
004  [ext_resource type="PackedScene" uid="uid://bokgju2g6vn6d" path="res://assetss/radio_v03.glb"
id="3_jyhfs"]
005  [ext_resource type="PackedScene" uid="uid://14y2qdh41q1d" path="res://assetss/Без имени.glb"
id="3_sugp2"]
006  [ext_resource type="Script" uid="uid://cr328nlnt4lhh" path="res://scripts/radio.gd" id="4_tbgi4"]
007
008  [sub_resource type="Environment" id="Environment_rrmyu"]
009  background_mode = 1
010  background_color = Color(0.003, 0.006, 0.014, 1)
011  ambient_light_source = 2
012  ambient_light_color = Color(0.025, 0.045, 0.105, 1)
013  ambient_light_energy = 0.22
014  tonemap_mode = 3
015  tonemap_exposure = 0.82
016  glow_enabled = true
017  glow_intensity = 0.55
018  glow_bloom = 0.18
019
020  [sub_resource type="BoxShape3D" id="BoxShape3D_rrmyu"]
021
022  [node name="main" type="Node3D" unique_id=251093681]
023
024  [node name="Player" parent="." unique_id=502170894 instance=ExtResource("1_player_scene")]
025  transform = Transform3D(-0.034183204, 0, -0.9994156, 0, 1, 0, 0.9994156, 0, -0.034183204, 26.122066,
3.6158824, -5.49512)
026  speed = 3.0
027  gravity = 9.8
028  mouse_sensetivity = 0.003
029
030  [node name="WorldEnvironment" type="WorldEnvironment" parent="." unique_id=1658428371]
031  environment = SubResource("Environment_rrmyu")
032
033  [node name="Level" type="Node3D" parent="." unique_id=1186078414]
034
035  [node name="CeilingLamp_Cool" type="OmniLight3D" parent="Level" unique_id=1202004188]
036  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 26.657188, 4.8184924, -3.6540704)
037  light_color = Color(0.31, 0.48, 1, 1)
038  light_energy = 2.4
039  shadow_enabled = true
040  omni_range = 6.0
041
```

```
042  [node name="Без имени" parent="Level" unique_id=1053881562 instance=ExtResource("3_sugp2")]
043  transform = Transform3D(0.0007881463, 0, 1.4999998, 0, 1.5, 0, -1.4999998, 0, 0.0007881463, 26.655256,
0, -5.8057346)
044
045  [node name="ConsoleLamp_Amber" type="OmniLight3D" parent="Level" unique_id=1739422142]
046  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 26.568779, 4.852949, -9.417564)
047  light_color = Color(1, 0.24, 0.055, 1)
048  light_energy = 1.55
049  shadow_enabled = true
050  omni_range = 4.5
051
052  [node name="RadioLamp_Cyan" type="OmniLight3D" parent="Level" unique_id=1378978383]
053  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 27.55, 4.55, -6.45)
054  light_color = Color(0.05, 0.55, 0.95, 1)
055  light_energy = 7.384
056  light_indirect_energy = 2.301
057  omni_range = 2.5
058
059  [node name="AirlockEmergency_Red" type="OmniLight3D" parent="Level" unique_id=988453784]
060  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 26.62, 4.55, -11.45)
061  light_color = Color(1, 0.015, 0.02, 1)
062  light_energy = 1.1
063  shadow_enabled = true
064  omni_range = 3.2
065
066  [node name="PilotNavigationLamp" type="SpotLight3D" parent="Level" unique_id=1058207634]
067  transform = Transform3D(1, 0, 0, 0, -0.5692547, 0.82216126, 0, -0.82216126, -0.5692547, 26.704636,
4.7404056, -0.43126452)
068  light_color = Color(0.050980393, 0.54901963, 0.87058824, 1)
069  light_energy = 10.785
070  light_indirect_energy = 0.862
071  shadow_enabled = true
072  spot_range = 6.673
073
074  [node name="radio_v03" parent="Level" unique_id=2074945346 instance=ExtResource("3_jyhfs")]
075  transform = Transform3D(0.0056547816, 0, -0.09983999, 0, 0.1, 0, 0.09983999, 0, 0.0056547816, 28.364971,
3.2908094, -5.2501903)
076
077  [node name="StaticBody3D" type="StaticBody3D" parent="Level/radio_v03" unique_id=1102499517]
078  script = ExtResource("4_tbgi4")
079
080  [node name="CollisionShape3D" type="CollisionShape3D" parent="Level/radio_v03/StaticBody3D"
unique_id=1885463891]
081  transform = Transform3D(7.0522738, 0, 5.9604645e-08, 0, 5.3357434, 0, -2.9802322e-08, 0, 3.6538517, -
1.428772, 3.5760727, -4.854431)
082  shape = SubResource("BoxShape3D_rrmyu")
083
084  [node name="CameraTarget" type="Camera3D" parent="Level/radio_v03" unique_id=542571916]
085  transform = Transform3D(9.999905, -0.0062609985, 0.04318136, 0, 9.896513, 1.4349262, -0.043632865, -
1.4349124, 9.89642, -1.597084, 4.0445786, -0.009307861)
086
087  [editable path="Level/radio_v03"]
```

## Б 4 Главное меню

Файл scenes/main_menu.tscn. Листинг содержит материал Земли, экранный шейдер, анимацию положения камеры, кнопки и соединения pressed и animation_finished.

```
001  [gd_scene format=3 uid="uid://chp56y0i0a2wj"]
002
003  [ext_resource type="PackedScene" uid="uid://cw6kf0fd2kjqu" path="res://assetss/earth.glb" id="1_bqqt6"]
004  [ext_resource type="Script" uid="uid://bsgsk2juvsh5y" path="res://scripts/main_menu.gd" id="1_rtw2f"]
005  [ext_resource type="Texture2D" uid="uid://f5lqt5kg5qws" path="res://assetss/earth_8081_earthmap10k.jpg"
id="2_wu84c"]
006  [ext_resource type="Texture2D" uid="uid://g6r2jpn3d413" path="res://assetss/earth_8081_earthspec10k.jpg"
id="3_8ln24"]
007  [ext_resource type="Script" uid="uid://nyqymlm2i5l5" path="res://scripts/earthrotating.gd" id="4_wu84c"]
008  [ext_resource type="Shader" uid="uid://bii4geh356ypj" path="res://shaders/dither.gdshader.gdshader"
id="5_8ln24"]
009
010  [sub_resource type="StandardMaterial3D" id="StandardMaterial3D_vue75"]
011  albedo_texture = ExtResource("2_wu84c")
012  roughness_texture = ExtResource("3_8ln24")
```

```
013
014 [sub_resource type="StandardMaterial3D" id="StandardMaterial3D_rrmyu"]
015 albedo_color = Color(0, 0, 0, 1)
016
017 [sub_resource type="Environment" id="Environment_rrmyu"]
018 tonemap_mode = 3
019 glow_enabled = true
020 glow_intensity = 1.2
021 glow_bloom = 0.2
022 adjustment_enabled = true
023 adjustment_contrast = 1.2
024 adjustment_saturation = 1.3
025
026 [sub_resource type="ShaderMaterial" id="ShaderMaterial_oa1go"]
027
028 [sub_resource type="ShaderMaterial" id="ShaderMaterial_rrmyu"]
029 shader = ExtResource("5_8ln24")
030 shader_parameter/dither_size = 4.0
031 shader_parameter/color_depth = 12.0
032
033 [sub_resource type="Animation" id="Animation_rtw2f"]
034 length = 0.001
035 tracks/0/type = "value"
036 tracks/0/imported = false
037 tracks/0/enabled = true
038 tracks/0/path = NodePath("Camera3D:position")
039 tracks/0/interp = 1
040 tracks/0/loop_wrap = true
041 tracks/0/keys = {
042 "times": PackedFloat32Array(0),
043 "transitions": PackedFloat32Array(1),
044 "update": 0,
045 "values": [Vector3(-0.20875561, -0.6954788, -0.51501644)]
046 }
047
048 [sub_resource type="Animation" id="Animation_oa1go"]
049 resource_name = "new_animation"
050 tracks/0/type = "value"
051 tracks/0/imported = false
052 tracks/0/enabled = true
053 tracks/0/path = NodePath("Camera3D:position")
054 tracks/0/interp = 1
055 tracks/0/loop_wrap = true
056 tracks/0/keys = {
057 "times": PackedFloat32Array(0, 1),
058 "transitions": PackedFloat32Array(1, 1),
059 "update": 0,
060 "values": [Vector3(-0.20875561, -0.6954788, -0.51501644), Vector3(-22.693539, 0.19174302, 1.5799751)]
061 }
062
063 [sub_resource type="AnimationLibrary" id="AnimationLibrary_oa1go"]
064 _data = {
065 &"RESET": SubResource("Animation_rtw2f"),
066 &"new_animation": SubResource("Animation_oa1go")
067 }
068
069 [node name="MainMenu" type="Node3D" unique_id=1004074282]
070 script = ExtResource("1_rtw2f")
071
072 [node name="earth" parent="." unique_id=906724121 instance=ExtResource("1_bqqt6")]
073 transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, -0.13354063, 0, -0.24444485)
074
075 [node name="Atmosphere" parent="earth" index="0" unique_id=455080335]
076 transform = Transform3D(0.97759277, 0, 0, 0, 0.9775927, 0, 0, 0, 0.97759277, -6.0492005, -0.33012927, -
0.99645257)
077 material_override = SubResource("StandardMaterial3D_vue75")
078 script = ExtResource("4_wu84c")
079
080 [node name="CSGBox3D" type="CSGBox3D" parent="." unique_id=1029735681]
081 transform = Transform3D(1, 0, 0, 0, 20.928196, 0, 0, 0, 44.44181, -23.62286, 0, 0)
082 material_override = SubResource("StandardMaterial3D_rrmyu")
083
084 [node name="Camera3D" type="Camera3D" parent="." unique_id=780494979]
085 transform = Transform3D(-0.09277185, 0.039088972, 0.9949198, 0, 0.9992291, -0.03925828, -0.99568737, -
0.0036420631, -0.09270033, -0.20875561, -0.6954788, -0.51501644)
086 fov = 31.3
087
```

```
088  [node name="DirectionalLight3D" type="DirectionalLight3D" parent="." unique_id=1800124558]
089  transform = Transform3D(1, 0, 0, 0, 0.93503475, 0.35455602, 0, -0.35455602, 0.93503475, -1.7809925,
2.8909163, 4.3157034)
090  light_energy = 0.5
091
092  [node name="WorldEnvironment" type="WorldEnvironment" parent="." unique_id=1403606107]
093  environment = SubResource("Environment_rrmyu")
094
095  [node name="CanvasLayer" type="CanvasLayer" parent="." unique_id=1230015142]
096
097  [node name="MenuButtons" type="VBoxContainer" parent="CanvasLayer" unique_id=1604858925]
098  modulate = Color(0.84325075, 0.91676646, 0.9218222, 1)
099  self_modulate = Color(0.1681245, 0.4507441, 0.32387736, 1)
100  material = SubResource("ShaderMaterial_oa1go")
101  offset_left = 42.0
102  offset_top = 95.00001
103  offset_right = 95.0
104  offset_bottom = 183.0
105  scale = Vector2(2.7558079, 2.6427143)
106
107  [node name="PlayButton" type="Button" parent="CanvasLayer/MenuButtons" unique_id=2111833331]
108  layout_mode = 2
109  theme_override_font_sizes/font_size = 7
110  text = "Новая игра"
111
112  [node name="Continue" type="Button" parent="CanvasLayer/MenuButtons" unique_id=782179366]
113  layout_mode = 2
114  theme_override_font_sizes/font_size = 7
115  text = "Продолжить"
116
117  [node name="Settings" type="Button" parent="CanvasLayer/MenuButtons" unique_id=344159659]
118  layout_mode = 2
119  theme_override_font_sizes/font_size = 7
120  text = "Настройки"
121
122  [node name="QuitButton" type="Button" parent="CanvasLayer/MenuButtons" unique_id=783514260]
123  layout_mode = 2
124  theme_override_font_sizes/font_size = 7
125  text = "Выйти"
126
127  [node name="ColorRect" type="ColorRect" parent="CanvasLayer" unique_id=561313235]
128  material = SubResource("ShaderMaterial_rrmyu")
129  anchors_preset = 15
130  anchor_right = 1.0
131  anchor_bottom = 1.0
132  grow_horizontal = 2
133  grow_vertical = 2
134  mouse_filter = 2
135
136  [node name="AnimationPlayer" type="AnimationPlayer" parent="." unique_id=605507256]
137  libraries/ = SubResource("AnimationLibrary_oa1go")
138
139  [connection signal="pressed" from="CanvasLayer/MenuButtons/PlayButton" to="."
method="_on_play_button_pressed"]
140  [connection signal="pressed" from="CanvasLayer/MenuButtons/QuitButton" to="."
method="_on_quit_button_pressed"]
141  [connection signal="animation_finished" from="AnimationPlayer" to="."
method="_on_animation_player_animation_finished"]
142
143  [editable path="earth"]
```

## Б 5 Сцена вступления

Файл scenes/intro_cutscene.tscn. По нему можно проверить имена RESET и new_animation, одинаковые ключевые позиции камеры и отсутствие звуковых дорожек в анимации.

```
001  [gd_scene format=3 uid="uid://byjj8bvykid88"]
002
003  [ext_resource type="Script" uid="uid://dsqrkn30mjlnp" path="res://scripts/intro_cutscene.gd"
id="1_dw6o5"]
004  [ext_resource type="Shader" uid="uid://bii4geh356ypj" path="res://shaders/dither.gdshader.gdshader"
id="2_3438a"]
```

```
005  [ext_resource type="AudioStream" uid="uid://dw3j2xxhfdewg" path="res://assetss/the-sound-of-a-spaceship-
taking-off-a-rocket.mp3" id="3_3438a"]
006  [ext_resource type="AudioStream" uid="uid://wnbfau8sfilq" path="res://assetss/vinilovoe-tploe-
poskripyivanie-ts100065.mp3" id="4_wifpd"]
007  [ext_resource type="PackedScene" uid="uid://14y2qdh41q1d" path="res://assetss/Без имени.glb"
id="6_jusdg"]
008
009  [sub_resource type="ShaderMaterial" id="ShaderMaterial_wifpd"]
010  shader = ExtResource("2_3438a")
011  shader_parameter/dither_size = 5.0
012  shader_parameter/color_depth = 12.0
013
014  [sub_resource type="Animation" id="Animation_dw6o5"]
015  length = 0.001
016  tracks/0/type = "value"
017  tracks/0/imported = false
018  tracks/0/enabled = true
019  tracks/0/path = NodePath("Camera3D:position")
020  tracks/0/interp = 1
021  tracks/0/loop_wrap = true
022  tracks/0/keys = {
023  "times": PackedFloat32Array(0),
024  "transitions": PackedFloat32Array(1),
025  "update": 0,
026  "values": [Vector3(26.774445, 3.8127213, 0.11605072)]
027  }
028
029  [sub_resource type="Animation" id="Animation_xeku4"]
030  resource_name = "new_animation"
031  length = 8.366667
032  tracks/0/type = "value"
033  tracks/0/imported = false
034  tracks/0/enabled = true
035  tracks/0/path = NodePath("Camera3D:position")
036  tracks/0/interp = 1
037  tracks/0/loop_wrap = true
038  tracks/0/keys = {
039  "times": PackedFloat32Array(0, 0.05),
040  "transitions": PackedFloat32Array(1, 1),
041  "update": 0,
042  "values": [Vector3(26.774445, 3.8127213, 0.11605072), Vector3(26.774445, 3.8127213, 0.11605072)]
043  }
044
045  [sub_resource type="AnimationLibrary" id="AnimationLibrary_xeku4"]
046  _data = {
047  &"RESET": SubResource("Animation_dw6o5"),
048  &"new_animation": SubResource("Animation_xeku4")
049  }
050
051  [sub_resource type="BoxShape3D" id="BoxShape3D_dcu1r"]
052
053  [node name="Node3D" type="Node3D" unique_id=894468114]
054  script = ExtResource("1_dw6o5")
055
056  [node name="CanvasLayer" type="CanvasLayer" parent="." unique_id=191105281]
057
058  [node name="ColorRect" type="ColorRect" parent="CanvasLayer" unique_id=451619795]
059  material = SubResource("ShaderMaterial_wifpd")
060  anchors_preset = 15
061  anchor_right = 1.0
062  anchor_bottom = 1.0
063  grow_horizontal = 2
064  grow_vertical = 2
065  mouse_filter = 2
066
067  [node name="AnimationPlayer" type="AnimationPlayer" parent="." unique_id=1848551459]
068  libraries/ = SubResource("AnimationLibrary_xeku4")
069
070  [node name="EngineSound" type="AudioStreamPlayer" parent="." unique_id=1108775630]
071  stream = ExtResource("3_3438a")
072  volume_db = 3.902
073
074  [node name="Tick" type="AudioStreamPlayer" parent="." unique_id=2021756126]
075  stream = ExtResource("4_wifpd")
076
077  [node name="CeilingLamp_Cool" type="OmniLight3D" parent="." unique_id=710402292]
078  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 26.657188, 4.8184924, -3.6540704)
```

```
079  light_color = Color(0.31, 0.48, 1, 1)
080  light_energy = 2.4
081  shadow_enabled = true
082  omni_range = 6.0
083
084  [node name="Без имени" parent="." unique_id=1622101041 instance=ExtResource("6_jusdg")]
085  transform = Transform3D(0.0007881463, 0, 1.4999998, 0, 1.5, 0, -1.4999998, 0, 0.0007881463, 26.655256,
0, -5.8057346)
086
087  [node name="ConsoleLamp_Amber" type="OmniLight3D" parent="." unique_id=423278322]
088  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 26.568779, 4.852949, -9.417564)
089  light_color = Color(1, 0.24, 0.055, 1)
090  light_energy = 1.55
091  shadow_enabled = true
092  omni_range = 4.5
093
094  [node name="RadioLamp_Cyan" type="OmniLight3D" parent="." unique_id=62839412]
095  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 27.55, 4.55, -6.45)
096  light_color = Color(0.05, 0.55, 0.95, 1)
097  light_energy = 7.384
098  light_indirect_energy = 2.301
099  omni_range = 2.5
100
101  [node name="AirlockEmergency_Red" type="OmniLight3D" parent="." unique_id=1041222623]
102  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 26.62, 4.55, -11.45)
103  light_color = Color(1, 0.015, 0.02, 1)
104  light_energy = 1.1
105  shadow_enabled = true
106  omni_range = 3.2
107
108  [node name="PilotNavigationLamp" type="SpotLight3D" parent="." unique_id=227444900]
109  transform = Transform3D(1, 0, 0, 0, -0.5692547, 0.82216126, 0, -0.82216126, -0.5692547, 26.704636,
4.7404056, -0.43126452)
110  light_color = Color(0.050980393, 0.54901963, 0.87058824, 1)
111  light_energy = 10.785
112  light_indirect_energy = 0.862
113  shadow_enabled = true
114  spot_range = 6.673
115
116  [node name="CollisionShape3D" type="CollisionShape3D" parent="." unique_id=1236479412]
117  transform = Transform3D(7.0522738, 0, 5.9604645e-08, 0, 5.3357434, 0, -2.9802322e-08, 0, 3.6538517, -
1.428772, 3.5760727, -4.854431)
118  shape = SubResource("BoxShape3D_dcu1r")
119
120  [node name="Camera3D" type="Camera3D" parent="." unique_id=427236518]
121  transform = Transform3D(-0.9799248, 0.18452266, 0.07549014, 0, 0.3786483, -0.9255406, -0.19936743, -
0.9069602, -0.37104687, 26.774445, 3.8127213, 0.11605072)
122
123  [connection signal="animation_finished" from="AnimationPlayer" to="."
method="_on_animation_player_animation_finished"]
```

## Б 6 Вспомогательная сцена

Файл scenes/test_scene.tscn. Сцена содержит CSGBox3D и CharacterBody3D со скриптом игрока, но не содержит полного набора дочерних узлов, требуемых player.gd.

```
001  [gd_scene format=3 uid="uid://40bv6wql7egk"]
002
003  [ext_resource type="Script" uid="uid://d3cpik1ayfhsl" path="res://scripts/player.gd" id="1_g7g4h"]
004
005  [node name="Node3D" type="Node3D" unique_id=1056321706]
006
007  [node name="CSGBox3D" type="CSGBox3D" parent="." unique_id=1281116086]
008  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, 0, -0.1407268, 0)
009  size = Vector3(1, 0.2944336, 1)
010
011  [node name="Player" type="CharacterBody3D" parent="." unique_id=1859687673]
012  transform = Transform3D(1, 0, 0, 0, 1, 0, 0, 0, 1, -0.30362457, 0, 0)
013  script = ExtResource("1_g7g4h")
014  metadata/_custom_type_script = "uid://d3cpik1ayfhsl"
```

## Б 7 Звуковая шина

Файл assetss/default_bus_layout.tres. Здесь сохранены громкость главной шины и включенный AudioEffectFilter.

```
001  [gd_resource type="AudioBusLayout" format=3 uid="uid://doeotxymw54lm"]
002
003  [sub_resource type="AudioEffectFilter" id="AudioEffectFilter_ba1sp"]
004  resource_name = "Filter"
005
006  [resource]
007  bus/0/volume_db = -2.312501
008  bus/0/effect/0/effect = SubResource("AudioEffectFilter_ba1sp")
009  bus/0/effect/0/enabled = true
```

## Приложение В Перечень скриншотов

В основной части размещены девять пронумерованных иллюстраций, содержащих одиннадцать кадров проекта. Скриншоты получены непосредственно из области отображения Godot при загрузке существующих сцен. Для обзорных видов использована диагностическая камера, для игровых действий — камера персонажа. В сравнительных кадрах менялась только видимость фонаря или экранного слоя шейдера.

| Номер | Что показать | Место вставки |
|---|---|---|
| 1 | Обзор основной сцены с контрастным светом | Раздел 3 |
| 2 | Обратный ракурс интерьера | Раздел 4 |
| 3 | Главное меню с Землей и кнопками | Раздел 5 |
| 4 | Текущий кадр вступления с дизерингом | Раздел 6 |
| 5 | Вид камеры персонажа после приземления | Раздел 7 |
| 6 | Крупный план радио после перехода камеры | Раздел 9 |
| 7 | Тот же прибор после команды D | Раздел 9 |
| 8 | Два кадра с выключенным и включенным фонарем | Раздел 10 |
| 9 | Два кадра меню с шейдером и без него | Раздел 11 |

Главное меню снималось отдельным запуском main_menu.tscn, поскольку главной сценой проекта остается main.tscn. Изображение вступления показывает текущее положение камеры и существующий экранный эффект, включая ограничение незавершенной анимации. Крупные планы радио сняты после завершения перехода камеры к прибору.

Пары кадров фонаря и дизеринга сняты с одинаковым ракурсом внутри каждой пары. При сравнении шейдера вращение Земли временно остановлено в диагностическом экземпляре, чтобы различие относилось к экранному эффекту. Исходные игровые сцены и скрипты при подготовке иллюстраций не изменялись.

## Приложение Г Перечень исходных материалов

Основной источник описания — текущие файлы GodotGame-main. Требования к полной игре изложены в документе «Постановка задачи ECHOES OF THE VOID.docx». Схемы процессов находятся в diagrams_echoes_of_the_void.drawio, activity_diagram_drawio.xml, bpmn_diagram_drawio.xml, activity_start_new_game.xml, activity_radio_signal.xml и bpmn_eva_repair.xml.

Исполняемую логику определяют шесть файлов каталога scripts. Иерархии сцен и их параметры определяют пять TSCN каталога scenes. Визуальный эффект определяет shaders/dither.gdshader.gdshader. Настройки приложения определяет project.godot, а конфигурацию звука — assetss/default_bus_layout.tres. Трехмерные, графические и звуковые ресурсы находятся в assetss; исходные файлы Blender — в blender.

В истории Git имеются два коммита от 17 сентября 2026 года: e049c7f с начальным проектом Godot и 38e26b2 с добавлением схем. История подтверждает сохранение проекта и документации в системе контроля версий. Дата коммита не устанавливает время создания каждого ресурса или длительность разработки отдельных механик.